
TVGL–CFM: Generating and Forecasting Time-Varying Trajectories of Dynamic Networks with Conditional Flow Matching

Om Roy*

University of Strathclyde
o.roy.2022@uni.strath.ac.uk

Yashar Moshfeghi

University of Strathclyde
yashar.moshfeghi@strath.ac.uk

Keith Malcolm Smith

University of Strathclyde
keith.smith@strath.ac.uk

Abstract

Many complex systems, including brain networks, financial markets, and gene-regulatory circuits, are better described by interaction structures that evolve over time than by a single fixed graph. The time-varying graphical lasso (TVGL) estimates this structure from multivariate signals as a temporally coherent sequence of sparse precision matrices. We introduce **TVGL–CFM**, a unified generative framework that learns distributions over complete SPD precision-matrix trajectories without requiring a pre-specified graph, supporting both class-conditional generation and history-conditioned forecasting. An SPD trajectory with T windows lies on the product Riemannian manifold $(\mathbb{S}_{++}^p)^T$, and we construct a global log-Euclidean diffeomorphism from this product space to a Euclidean sequence space, letting a non-autoregressive conditional flow-matching model with a Transformer backbone generate all windows jointly and decode to SPD matrices with no post-hoc projection. For forecasting, we use two distinct data dependant couplings so the flow transforms an informative prior into a coherent future block. Across EEG motor-imagery data and three nonlinear dynamical systems, TVGL–CFM preserves class-discriminative dependency structure and forecasts future connectivity more accurately than several strongly matched baselines opening up new possibilities for generative dynamic graph models.

1 Introduction

A wide range of systems are most naturally described not by a static network but by a network whose dependency structure changes over time: functional brain connectivity reconfigures within a task or under stimulation, and cross-asset dependence in financial markets shifts across regimes. When the signal at each node is (locally) approximately Gaussian, the canonical descriptor of such structure is the precision matrix (inverse covariance), whose zeros encode conditional independence. Estimating it is the graphical-lasso problem [2–4], whose time-varying extension, the time-varying graphical lasso (TVGL) [1], estimates a chain of sparse precision matrices that is both sparse at each time point and temporally coherent across time. When there is no known underlying graph, which is common in various important applications, TVGL provides a robust approach that produces plausible graph trajectories.

Diffusion models are a strong generative paradigm for complex data distributions [29, 31, 33], and have been adapted to structured scientific objects such as molecules, materials [50] and proteins [52]; but they have practical costs. Sampling typically needs many iterative denoising steps, and the stochastic reverse process is expensive when the generated object is not a simple Euclidean vector but a structured trajectory, graph, or matrix-valued object.

*Corresponding author.

Conditional flow matching keeps the strengths of diffusion-style modelling while simplifying training and sampling [22, 25, 26]. Instead of denoising through a long stochastic reverse chain, flow matching learns a time-dependent vector field that transports samples from a simple source distribution to the target along a continuous-time ODE [28]. In the conditional setting this transport can be guided by context, labels, or partial trajectories, which makes it attractive for forecasting as the model can generate an entire future trajectory jointly, avoiding autoregressive error accumulation and long sampling chains.

Trajectories of precision matrices lie on a Riemannian manifold rather than in Euclidean space, which makes flow matching costly: it requires minimising along geodesics rather than straight-line interpolants. Diffeomorphic flow matching bypasses this by mapping the manifold through a global diffeomorphic chart, so that ordinary Euclidean flow-matching machinery can be used. In particular, DIFFEOCFM [45] applied these diffeomorphisms to brain connectivity matrices which are usually represented as covariance or correlation matrices. We extend this idea from a single matrix to a *trajectory* of precision matrices, and show that the log-Euclidean chart is an exact diffeomorphism on the product manifold. This yields an efficient paradigm, which we call TVGL-CFM: a generative model for time-varying dynamical graphs, estimated directly from multivariate time series.

Two capabilities are valuable once such trajectories can be obtained. The first is **generation**: sampling new, realistic, class-conditional trajectories, which is useful when real recordings are scarce, as they often are in clinical neuroimaging [45, 76]. The second is **forecasting**: given an observed prefix, predicting how the trajectory continues, for example how a brain network will evolve in the windows following a stimulation pulse [71, 72]. We build on the geometric insight and extend it in two directions that, to our knowledge, have not been combined before: from a single matrix to a whole trajectory of matrices evolving over time, and from *generation* to *forecasting* a trajectory’s future from its past. We extend trajectory flow matching to history-conditioned forecasting of an entire future precision trajectory, studying both a mean-centred residual formulation and a random-walk stochastic source. Against baselines that generate or forecast raw EEG and estimate connectivity afterwards, modelling the precision trajectory directly is markedly better on both tasks; against models trained on the identical TVGL representation the margin is smaller but TVGL-CFM is still the best performing, outperforming even a strong dynamic graph transformer architecture modified to process TVGL representations. We further show transferability to a different temporal graphical model providing evidence that our framework can be used as a generalized tool.

Contributions

- **Flow matching over dynamic networks without a pre-specified graph.** We recover the latent dynamic network as a trajectory of time-varying sparse precision matrices with TVGL and learn to generate and forecast these trajectories directly, rather than modelling raw signals and estimating connectivity afterwards. For forecasting we study two data-dependent couplings: a mean-centred residual coupling, which models Gaussian residuals around a training-set future mean while conditioning the velocity field on the observed history, and a random-walk coupling whose stochastic source is itself built from recent increments of the observed trajectory.
- **A transformer-based, non-autoregressive flow-matching model on the SPD manifold.** A global log-Euclidean chart flattens an entire precision trajectory into a Euclidean coordinate, where one transformer velocity field, sampled non-autoregressively over the whole sequence, drives a single conditional flow-matching model and decodes to guaranteed-SPD matrices with no post-hoc projection.
- **Competitive generation and forecasting across domains.** The same geometric CFM framework supports class-conditional generation $p(\Theta_{1:T} \mid y)$ and history-conditioned forecasting $p(\Theta_{K+1:T} \mid \Theta_{1:K}, y)$, outperforming raw-signal baselines on every task and leading or matching models trained directly on the same precision trajectories.

2 Background

2.1 SPD matrices and the log-Euclidean diffeomorphism

Let $\mathbb{S}_{++}^p = \{\Sigma \in \mathbb{R}^{p \times p} \mid \Sigma^\top = \Sigma, \Sigma \succ 0\}$. The matrix logarithm composed with half-vectorisation,

$$\phi(\Sigma) = \text{vec}_{\text{lt}}(\log \Sigma), \quad \phi^{-1}(\eta) = \exp(\text{vec}_{\text{lt}}^{-1}(\eta)), \quad (1)$$

is a global diffeomorphism from $\mathbb{S}_{++}^p$ onto $\mathbb{R}^{p(p+1)/2}$, where vec_{lt} stacks the lower-triangular entries with off-diagonal terms scaled by $\sqrt{2}$ [55, 56]. Declaring ϕ distance-preserving defines the *log-Euclidean* metric on $\mathbb{S}_{++}^p$; equivalently, the Euclidean inner product on $\mathbb{R}^{p(p+1)/2}$ is *pulled back* through ϕ to a Riemannian metric on $\mathbb{S}_{++}^p$. It is one of several structures proposed for SPD and correlation matrices (Appendix A), and the one under which flow matching reduces to a Euclidean problem. Because $\Theta = \Sigma^{-1}$ has reciprocal eigenvalues and identical eigenvectors,

$$\phi(\Theta) = -\phi(\Sigma), \quad (2)$$

so flowing precision in this embedding is equivalent to flowing covariance up to a sign, and discriminative information is preserved under the map.

2.2 Conditional flow matching and diffeomorphic transport

Flow matching learns a time-dependent velocity field $u_\theta(s, z)$ that transports samples from a simple source distribution $p_0 = \mathcal{N}(0, \mathbf{I})$ to a target distribution p_1 along an ODE $\dot{z} = u_\theta(s, z)$ [22, 25, 26], where $s \in [0, 1]$ is the *flow time*. With the linear conditional path $z_s = (1-s)z_0 + sz_1$ and target velocity $z_1 - z_0$, the CFM loss is

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s, z_0 \sim p_0, z_1 \sim p_1} \|u_\theta(s, (1-s)z_0 + sz_1) - (z_1 - z_0)\|_2^2. \quad (3)$$

Riemannian flow matching extends this to data on a manifold $\mathcal{M}$ [24], but applying it directly to SPD precision trajectories requires manifold-valued paths and Riemannian vector fields, at higher cost [47, 54]. Diffeomorphic flow matching avoids this with a global diffeomorphism $\phi : \mathcal{M} \rightarrow E$ into a Euclidean space, along whose pullback path the Riemannian CFM loss is exactly the Euclidean CFM loss in the embedded coordinates $z = \phi(x)$ [45]. For TVGL-CFM, $\mathcal{M}$ is the product SPD trajectory space $\mathcal{M}_T = (\mathbb{S}_{++}^p)^T$ and ϕ is the log-Euclidean embedding applied window-wise; Section 3.2 makes this precise.

2.3 Time-varying graphical lasso

Given a multivariate signal windowed into T slices with empirical covariances $S_1, \dots, S_T$ (each from n samples), TVGL [1] estimates a chain of sparse precision matrices $\Theta = (\Theta_1, \dots, \Theta_T)$ by solving

$$\min_{\Theta_1, \dots, \Theta_T \in \mathbb{S}_{++}^p} \sum_{i=1}^T \left(-\log \det \Theta_i + \text{tr}(S_i \Theta_i) \right) + \lambda \sum_{i=1}^T \|\Theta_i\|_{\text{od},1} + \beta \sum_{i=2}^T \psi(\Theta_i - \Theta_{i-1}), \quad (4)$$

where $\|\cdot\|_{\text{od},1}$ is the off-diagonal ℓ_1 norm and ψ is a convex evolutionary penalty controlling how the network may change between adjacent timestamps. Following Hallac et al. [1] we consider three options for ψ , each encoding a different assumption about how the network reconfigures between windows: writing $D = \Theta_i - \Theta_{i-1}$, the element-wise ℓ_1 penalty $\|D\|_1$, the Laplacian penalty $\|D\|_F^2$, and the column group-lasso penalty $\sum_j \|D_{:,j}\|_2$. TVGL generalises static and jointly-estimated sparse graphical models [2, 5, 6] to an explicitly temporal chain; proximal operators and precursors are in Appendices A and C.

3 Method: TVGL-CFM

Problem setup. A dataset is a collection of p -channel sequences with optional class labels y . Each sequence is windowed into T slices and reduced, per window, to a TVGL precision matrix, giving a per-sequence trajectory $\Theta_{1:T} \in (\mathbb{S}_{++}^p)^T$. We address two tasks on these trajectories: *generation* of a full trajectory given a class, $p(\Theta_{1:T} | y)$, and *forecasting* of the future given an observed prefix of length K , $p(\Theta_{K+1:T} | \Theta_{1:K}, y)$. Both share the target-construction and embedding stages and differ only in the conditioning of the flow (Figures 1 and 2, Appendix B).

3.1 TVGL targets, embedding and standardisation

Each sequence’s p -channel signal is z -scored per channel over time and split into T contiguous windows of length n . Because n may be comparable to p , the raw sample covariance is poorly conditioned; we use the oracle-approximating-shrinkage (OAS) estimator [21], and divide every estimate by the mean training diagonal so that the TVGL λ is interpretable. We then solve (4) for all sequences and windows simultaneously with ADMM [18, 20], whose updates have closed forms in terms of proximal operators [19]. Appendix C gives the derivation and Algorithm 1; the output is a sparse SPD precision trajectory per sequence.

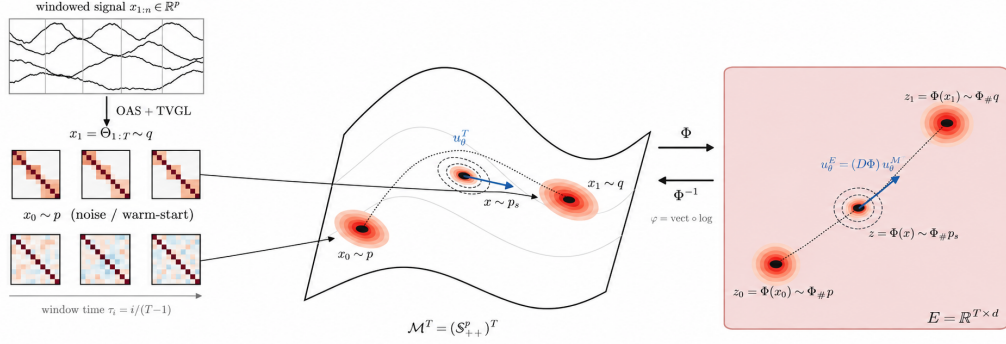


Figure 1: Flow matching on TVGL precision-matrix trajectories via log-Euclidean embedding.

A multivariate windowed signal $x_{1:n} \in \mathbb{R}^p$ is converted into a time-varying graphical trajectory $\Theta_{1:T}$ using OAS covariance estimation followed by TVGL, defining samples from the target distribution q on the product SPD manifold $\mathcal{M}^T = (\mathbb{S}_{++}^p)^T$. A learned Riemannian vector field $u_\theta^{\mathcal{M}^T}$ transports a source trajectory $z_0 \sim p$, implemented as Gaussian noise for generation, and under one of two data-dependent couplings for forecasting, toward a target $z_1 \sim q$. Trajectories are mapped through the log-Euclidean chart Φ into $E = \mathbb{R}^{T \times d}$, where the pushforwards are connected by $u_\theta^E = (D\Phi)u_\theta^{\mathcal{M}^T}$, enabling conditional flow matching while preserving the SPD structure after decoding with Φ^{-1} .

We embed each window of the target with the log-Euclidean diffeomorphism (1), applied independently across the trajectory:

$$\Phi(\Theta_{1:T}) = (\phi(\Theta_1), \dots, \phi(\Theta_T)) \in E = \mathbb{R}^{T \times d}, \quad d = p(p+1)/2. \quad (5)$$

These coordinates are not equally scaled: diagonal log-precision entries are typically much larger than off-diagonal edge entries, so training CFM directly on z_n would let high-variance coordinates dominate the squared loss. We therefore standardise each coordinate with training-set statistics (μ_k, σ_k) , an affine diffeomorphism $\Psi : E \rightarrow E$ composed with the chart to give $\tilde{\Phi} = \Psi \circ \Phi$:

$$\Psi(z)_{i,k} = \frac{z_{i,k} - \mu_k}{\sigma_k}, \quad \Psi^{-1}(\tilde{z})_{i,k} = \sigma_k \tilde{z}_{i,k} + \mu_k, \quad \tilde{z} = \tilde{\Phi}(\Theta_{1:T}). \quad (6)$$

The geometric validity of this embedding follows from the product structure of the trajectory manifold.

Proposition 1 (Product diffeomorphism and pullback metric). *$\Phi : \mathcal{M}_T \rightarrow E$ is a global diffeomorphism, and the product log-Euclidean metric (35) is the pullback $\Phi^* g_E$ of the standard Euclidean metric on E .*

Standardisation simply composes this product diffeomorphism with an invertible affine map.

Proposition 2 (Standardised diffeomorphism). *Assume $\sigma_k > 0$ for all $k = 1, \dots, d$. Then $\tilde{\Phi} := \Psi \circ \Phi : \mathcal{M}_T \rightarrow E$ is a global diffeomorphism. Its pullback metric is the per-coordinate rescaling*

$$(\tilde{\Phi}^* g_E)_{\Theta_{1:T}}(\xi, \eta) = \sum_{i=1}^T \sum_{k=1}^d \sigma_k^{-2} [D\phi(\Theta_i)[\xi_i]]_k [D\phi(\Theta_i)[\eta_i]]_k, \quad (7)$$

for tangent trajectories $\xi, \eta \in T_{\Theta_{1:T}} \mathcal{M}_T$.

Proofs are in Appendix E. Euclidean CFM in the standardised coordinates $\tilde{z} = \tilde{\Phi}(\Theta_{1:T})$ is therefore still a pullback Riemannian CFM model on the original SPD trajectory manifold, and the equivalences of Section 3.2 hold with $\tilde{\Phi}$ in place of Φ . After sampling we invert the standardisation and decode each window through the inverse log-Euclidean map:

$$\hat{\Theta}_i = \phi^{-1}(\sigma \odot \hat{\tilde{z}}_i + \mu), \quad i = 1, \dots, T. \quad (8)$$

Because ϕ^{-1} is the matrix exponential after inverse half-vectorisation, every decoded $\hat{\Theta}_i$ is SPD by construction. It is not, however, exactly sparse; but there are generated values close to 0 instead thus an optional sparsified readout is discussed in the Appendix (C.3). All metrics are computed on the unsparified output.

3.2 Generation: joint-trajectory conditional flow

A single transformer encoder u_θ [73] processes the whole standardised trajectory as a length- T token sequence, each token summed with three embeddings: flow time s , window time $\tau_i = i/(T-1)$ and class y , the continuous times entering through random Fourier features [74]. Writing $z \in \mathbb{R}^{T \times d}$,

$$u_\theta(z, s, y, \tau) = \text{Dec}\left(\text{TX}(\text{Enc}(z) + \text{FF}_s(s) + \text{FF}_\tau(\tau) + \text{Emb}_y(y))\right). \quad (9)$$

Here $\text{Enc} : \mathbb{R}^d \rightarrow \mathbb{R}^{d_{\text{model}}}$ is a linear token embedding applied per window, FF_s and FF_τ are random-Fourier-feature maps of the two continuous times followed by an MLP, Emb_y is a learned class embedding, TX is a Transformer encoder stack over the T window tokens, and $\text{Dec} : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^d$ is a linear read-out back to chart coordinates. Both CFM endpoints are whole trajectories: $E = \mathbb{R}^{T \times d} \cong \mathbb{R}^{Td}$, so $z_0 \sim \mathcal{N}(0, I_{Td})$ and $z_1 = \tilde{\Phi}(\Theta_{1:T})$ are single points in one trajectory-level space and $z_s = (1-s)z_0 + sz_1$ interpolates the whole trajectory at once. Because the standardised chart $\tilde{\Phi}$ is a global diffeomorphism onto E with pullback metric $\tilde{\Phi}^*g_E$ (Propositions 1–2), the three equivalences below hold with $\tilde{\Phi}$ in place of Φ ; we state them for Φ to lighten notation. The first shows that training the field in the embedding is exactly Riemannian CFM on the trajectory manifold. Throughout, the Euclidean field is the pushforward of a Riemannian field $u_\theta^{\mathcal{M}_T}$ on $(\mathcal{M}_T, \Phi^*g_E)$ through the differential $D\Phi$ (Eq. (10)).

Proposition 3 (Trajectory CFM loss equivalence). *Let p, q be class-conditional measures on $\mathcal{M}_T$ and write $z_0 \mid y \sim \Phi_{\#}p(\cdot \mid y)$, $z_1 \mid y \sim \Phi_{\#}q(\cdot \mid y)$ in E . Define the Euclidean field*

$$u_\theta^E(s, z, y) := D\Phi(\Phi^{-1}(z)) [u_\theta^{\mathcal{M}_T}(s, \Phi^{-1}(z), y)]. \quad (10)$$

*Then the Riemannian CFM loss on $(\mathcal{M}_T, \Phi^*g_E)$ equals*

$$\mathcal{L}_{\text{traj}}(\theta) = \mathbb{E}_{s, y, z_0 \mid y, z_1 \mid y} \|u_\theta^E(s, (1-s)z_0 + sz_1, y) - (z_1 - z_0)\|_E^2. \quad (11)$$

The proof is given in Appendix E.3.

The next result establishes that integrating the learned Euclidean ODE and decoding through Φ^{-1} reproduces the manifold-valued flow exactly.

Proposition 4 (Trajectory ODE equivalence). *Let $z : [0, 1] \rightarrow E$ solve $\dot{z}(s) = u_\theta^E(s, z(s), y)$ with $z(0) = z_0 = \Phi(\Theta_{1:T}^0)$. Then $\Theta_{1:T}(s) := \Phi^{-1}(z(s))$ solves the Riemannian ODE*

$$\dot{\Theta}_{1:T}(s) = u_\theta^{\mathcal{M}_T}(s, \Theta_{1:T}(s), y), \quad \Theta_{1:T}(0) = \Theta_{1:T}^0, \quad (12)$$

*on $(\mathcal{M}_T, \Phi^*g_E)$, and $\Theta_i(s) \in \mathbb{S}_{++}^p$ for every $i \in \{1, \dots, T\}$ and every $s \in [0, 1]$. The proof is given in Appendix E.4.*

The equivalence also holds at the discrete level: the same explicit Runge–Kutta scheme applied in E or on $\mathcal{M}_T$ produces iterates related by Φ , so the practical RK4 sampler of Algorithm 3 inherits the guarantees above.

Proposition 5 (Trajectory Runge–Kutta equivalence). *Let an explicit Runge–Kutta scheme with ν stages, Butcher tableau (a_{ij}, b_i, c_i) , and step size h be applied to the Riemannian ODE (12) on $(\mathcal{M}_T, \Phi^*g_E)$ and to the Euclidean ODE $\dot{z} = u_\theta^E(s, z, y)$ on E , with initial conditions $\Theta_{1:T}^0$ and $z^0 = \Phi(\Theta_{1:T}^0)$ respectively. Then*

$$\Theta_{1:T}^\ell = \Phi^{-1}(z^\ell) \quad \text{for all } \ell \in \mathbb{N}, \quad (13)$$

and every iterate $\Theta_{1:T}^\ell$ is slice-wise SPD. The proof is given in Appendix E.5.

TVGL temporal regulariser. For a path point z_s and predicted velocity u_θ , the implied clean endpoint is $\hat{z}_1 = z_s + (1-s)u_\theta$. Writing $\hat{z}_{1,i}$ for the i -th window slice of this clean endpoint, we bias the generator toward the chosen evolutionary pattern by penalising the corresponding ψ on consecutive windows of $\hat{z}_1$:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + \lambda_{\text{temp}} \mathbb{E} \left[\frac{1}{T-1} \sum_{i=2}^T \psi_E(\hat{z}_{1,i} - \hat{z}_{1,i-1}) \right], \quad (14)$$

with $\psi_E(\cdot) = \|\cdot\|_1$ for the ℓ_1 penalty and $\psi_E(\cdot) = \|\cdot\|_2^2$ for the smooth/group penalties. The sum is over window time at fixed flow time $s = 1$; the CFM term $\mathcal{L}_{\text{CFM}}$ itself averages over flow time. The main temporal structure is carried by the targets; (14) is a soft prior consistent with TVGL. Training and sampling are given in Algorithms 2–3.

Remark 1. Since $\Phi = \phi \times \dots \times \phi$ its differential is block diagonal, so the *chart* acts factor-wise. That concerns coordinates only: the components $u_{\theta,i}(\Theta_{1:T})$ of the learned field each depend on the *entire* trajectory through attention, so the geometry is a Cartesian product while the dynamics are globally coupled. A window-wise extension of DIFFOCFM [45] would transport each matrix independently, modelling roughly $\prod_t p(\Theta_t | y)$; we model $p(\Theta_{1:T} | y)$ directly.

3.3 Forecasting: history-conditioned future flow

We split each standardised trajectory into an observed history $z_{1:K}$ and future $z^f := z_{K+1:T}$, and learn a conditional flow over the future block. The model is non-autoregressive: it observes the prefix and samples the whole future jointly, so future windows attend to one another without autoregressive error accumulation.

History conditioning. A 2-layer Transformer encodes the prefix into one token per window,

$$H = \mathcal{E}_\theta(z_{1:K}) = \text{TX}_{\text{hist}}(\text{Enc}(z_{1:K}) + \text{FF}_\tau(\tau_{1:K}) + \text{Emb}_y(y)) \in \mathbb{R}^{K \times d_{\text{model}}}, \quad (15)$$

with Enc , FF_τ and Emb_y as in (9), omitting the last when no label is available. The future stack cross-attends to H , so each future window can attend to every observed window.

Source, path and field. As in generation, the endpoints are whole blocks in $E_f = \mathbb{R}^{(T-K) \times d} \cong \mathbb{R}^{(T-K)d}$. Writing z_1^f for the target at $s = 1$ and z_0^f for the source at $s = 0$,

$$z_s^f = (1-s)z_0^f + sz_1^f, \quad v_s^* = z_1^f - z_0^f, \quad (16)$$

and the field is

$$u_\theta(z_s^f, s, y, \tau_{K+1:T}, H) = \text{Dec}\left(\text{TX}_f(Z_s | H)\right), \quad (17)$$

$$Z_s = \text{Enc}(z_s^f) + \text{FF}_s(s) + \text{FF}_\tau(\tau_{K+1:T}) + \text{Emb}_y(y).$$

Here $\text{TX}_f(\cdot | H)$ self-attends over the $T - K$ future tokens and cross-attends to all history tokens in H . Section 3.4 specifies the coupling, i.e. z_0^f and z_1^f .

The field is trained with two additional losses, both evaluated on the absolute future $\hat{z}_1^f$ implied by the predicted velocity. The temporal regulariser (14) is applied to $[z_{1:K}; \hat{z}_1^f]$, enforcing coherence across the history–future handoff. We also match the first future transition:

$$\mathcal{L}_{\text{bnd}} = \mathbb{E} \left\| (\hat{z}_{1,1}^f - z_K) - (z_{1,1}^f - z_K) \right\|_2^2 = \mathbb{E} \left\| \hat{z}_{1,1}^f - z_{1,1}^f \right\|_2^2, \quad (18)$$

a squared log-Euclidean distance between the decoded matrices (Proposition 7). The total objective is

$$\mathcal{L}_{\text{forecast}} = \mathbb{E}_{(z_0^f, z_1^f) \sim \pi, y, s} \left\| u_\theta(z_s^f, s, y, \tau_{K+1:T}, H) - (z_1^f - z_0^f) \right\|_2^2 + \lambda_{\text{temp}} \mathcal{R}_{\text{temp}} + \lambda_{\text{bnd}} \mathcal{L}_{\text{bnd}}, \quad (19)$$

where π is the source–target coupling. Decoding inverts the standardisation and applies ϕ^{-1} window-wise, so every forecast is slice-wise SPD without projection.

3.4 Data-dependent couplings

The standard choice $z_0^f \sim \mathcal{N}(0, I)$, $z_1^f = z^f$ ignores information already contained in the history and training set. Guided flow matching instead uses an informative source so the flow models the remaining correction [81]. We therefore consider two source–target couplings π acting on opposite ends of the pair; Proposition 6 shows the CFM equivalence continues to hold for general couplings.

(a) Mean-centred residual coupling. From the training split only, define the mean future block in standardised coordinates,

$$m^f = \frac{1}{N} \sum_{n=1}^N z_{n,K+1:T} \in \mathbb{R}^{(T-K) \times d}, \quad (20)$$

and model deviations around it:

$$z_1^f = z^f - m^f, \quad z_0^f \sim \mathcal{N}(0, I). \quad (21)$$

Thus only the target is translated; the Gaussian source is unchanged, and (16)–(17) operate in residual coordinates. After integration, $\hat{z}^f = m^f + \hat{z}_1^f$, so a zero residual returns m^f .

(b) Random-walk coupling. Here the target remains absolute, $z_1^f = z^f$, while the source is a stochastic extrapolation of the observed prefix, following data-coupled trajectory simulation [83].

Table 1: Three-seed generation performance on EEG datasets. Entries are means over three random seeds; standard deviations are in Appendix M. CAS AUC and F1 measure train-on-generated $\rightarrow$ test-on-real classifiability (higher is better), while Rel-FD is the Fréchet distance between generated and held-out real precision trajectories normalized by the real train–test reference (lower is better). Direct precision-space models generate TVGL trajectories directly; raw-EEG baselines generate signals that are subsequently mapped through the same OAS+TVGL pipeline.

Method	Zhou2016			BNCI2014_001			BNCI2014_002			BNCI2015_001			Mean		
	AUC	F1	Rel-FD	AUC	F1	Rel-FD	AUC	F1	Rel-FD	AUC	F1	Rel-FD	AUC	F1	Rel-FD
TVGL oracle	0.965	0.900	–	0.794	0.722	–	0.777	0.717	–	0.762	0.692	–	–	–	–
TVGL–CFM	0.954	0.893	1.101	0.718	0.646	1.191	0.736	0.678	1.022	0.717	0.649	1.061	0.781	0.716	1.094
TVGL–cVAE	0.879	0.799	1.111	<u>0.677</u>	<u>0.620</u>	1.938	<u>0.680</u>	<u>0.621</u>	0.994	<u>0.693</u>	<u>0.635</u>	2.725	<u>0.732</u>	<u>0.669</u>	1.692
TVGL–DDPM	0.889	0.792	1.075	0.615	0.571	<u>1.572</u>	0.624	0.587	1.098	0.603	0.569	<u>1.223</u>	0.683	0.629	<u>1.242</u>
Window-DIFFEO-CFM	<u>0.933</u>	<u>0.853</u>	1.208	0.626	0.596	2.145	0.661	0.621	1.386	0.664	0.593	1.536	0.721	0.666	1.569
JET $\rightarrow$ TVGL	0.573	0.398	9.216	0.508	0.385	11.374	0.506	0.356	7.845	0.505	0.334	31.738	0.523	0.368	15.043
cVAE $\rightarrow$ TVGL	0.880	0.427	9.700	0.575	0.417	9.983	0.522	0.430	6.156	0.589	0.456	28.113	0.642	0.433	13.488
cWGAN-GP $\rightarrow$ TVGL	0.678	0.570	2.602	0.520	0.513	3.498	0.507	0.500	2.018	0.510	0.477	8.523	0.554	0.515	4.160
EEG-GAN $\rightarrow$ TVGL	0.375	0.335	8.288	0.537	0.373	12.802	0.547	0.391	4.474	0.566	0.417	5.792	0.507	0.379	7.839

From the last r increments, estimate elementwise drift and spread,

$$\hat{\mu} = \frac{1}{r} \sum_{j=K-r+1}^K (z_j - z_{j-1}), \quad \hat{\sigma} = \sqrt{\frac{1}{r} \sum_{j=K-r+1}^K (z_j - z_{j-1} - \hat{\mu})^{\odot 2}} + \sigma_{\min}, \quad (22)$$

then, from $\tilde{z}_{0,0}^f = z_K$, sample

$$\tilde{z}_{0,q}^f = \tilde{z}_{0,q-1}^f + \alpha_{\mu} \hat{\mu} + \alpha_{\sigma} \hat{\sigma} \odot \epsilon_q, \quad \epsilon_q \sim \mathcal{N}(0, I), \quad q = 1, \dots, T - K. \quad (23)$$

We set

$$z_0^f = (\tilde{z}_{0,1}^f, \dots, \tilde{z}_{0,T-K}^f) \sim p_{\text{RW}}(\cdot \mid z_{1:K}).$$

This random walk evolves through graph-state space: each z_j represents an entire precision matrix via $z_j = \phi(\Theta_j)$, so decoding any state remains SPD. Setting the informed source to plain Gaussian recovers the standard CFM construction.

4 Experiments

We evaluate on four MOABB motor-imagery datasets [70] (BNCI2014_001, BNCI2014_002, BNCI2015_001, Zhou2016) and on three simulated systems that probe domain generality: **Lorenz**, diffusively coupled Lorenz oscillators; **MacArthur**, a consumer-resource competition system; and **Hopfield**, a continuous associative-memory network. Preprocessing, architecture, baselines and metrics are in Appendices J–L. Every result is a mean over three seeds. Each sample is the whole chain (for forecasting, the future block), embedded per window and concatenated into a single vector, so metrics are sensitive to temporal arrangement.

4.1 Generating Realistic Brain-Network Dynamics

We assess downstream utility using the classification-accuracy score (CAS), training a small MLP on generated trajectories and evaluating on held-out real trials [84]. Generation quality is measured by relative Fréchet distance (Rel-FD) in the log-Euclidean embedding.

TVGL–CFM is strongest overall, achieving the highest CAS AUC/F1 on all four datasets and the best mean AUC, F1 and Rel-FD (0.781, 0.716, 1.094). TVGL–DDPM attains a slightly lower Rel-FD on Zhou2016 (1.075 vs. 1.101), and TVGL–cVAE on BNCI2014_002 (0.994 vs. 1.022).

The gap is much larger against raw-EEG generators decoded through TVGL: TVGL–CFM gains 0.074–0.189 AUC per dataset, with Rel-FD 1.022–1.191 versus 2.018–31.738. JET [30] reaches only 0.523 mean AUC at Rel-FD 15.043, indicating that realistic signal generation alone does not preserve temporal dynamics. Against models trained on the same TVGL trajectory representation, the margin is smaller but consistent: TVGL–CFM leads TVGL–cVAE, TVGL–DDPM and Window-DIFFEO-CFM in AUC on every dataset and in mean Rel-FD (1.094 vs. 1.692, 1.242, 1.569). Window-DIFFEO-CFM shares the targets and chart but removes temporal communication, dropping to 0.721 mean AUC at Rel-FD 1.569, supporting joint trajectory modelling. The mean AUC gap to the TVGL oracle is only 0.044, while every generated matrix remains SPD by construction.

Table 2: Three-seed forecasting performance on EEG datasets. Entries are means over three random seeds; standard deviations are in Appendix M. Lower is better for AIRM and log-Euclidean RMSE (logE). Both TVGL–CFM couplings are reported: the mean-centred residual coupling and the random-walk coupling. The remaining learned forecasters predict the absolute future directly. Best in **bold**, second best *underlined and italicized*.

Method	Zhou2016		BNCI2014_001		BNCI2014_002		BNCI2015_001		Mean	
	AIRM	logE	AIRM	logE	AIRM	logE	AIRM	logE	AIRM	logE
TVGL–CFM (mean-residual)	2.415	2.300	2.747	2.544	1.552	1.417	<u>1.952</u>	<u>1.793</u>	2.167	2.014
TVGL–CFM (Random-Walk)	2.937	2.773	3.445	3.149	1.817	1.636	2.167	1.979	2.591	2.384
TVGL–cVAE-Forecast	<u>2.452</u>	<u>2.337</u>	<u>2.706</u>	2.503	1.601	1.456	1.977	1.809	2.184	2.026
TVGL–DDPM-Forecast	2.768	2.687	3.361	3.206	1.797	1.660	2.140	2.000	2.517	2.388
DyGraphformer-T	2.460	2.348	2.696	<u>2.507</u>	<u>1.578</u>	<u>1.441</u>	1.940	1.780	<u>2.168</u>	<u>2.019</u>
Training mean (no flow)	2.847	2.769	3.286	3.143	1.839	1.707	2.203	2.067	2.544	2.421
Persistence	3.011	2.821	3.362	3.046	1.931	1.720	2.520	2.272	2.706	2.465
Basic-CNN → TVGL	9.843	9.884	10.897	10.943	5.920	5.946	8.739	8.764	8.850	8.884
cVAE → TVGL	8.392	8.422	9.098	9.120	3.143	3.097	6.046	6.029	6.670	6.667
cWGAN-GP → TVGL	9.809	9.851	10.735	10.780	4.902	4.925	5.342	5.279	7.697	7.709
EEG-GAN → TVGL	4.327	4.101	5.117	4.742	2.978	2.773	3.193	2.972	3.904	3.647
JET → TVGL	3.861	3.490	4.126	3.891	3.946	3.059	4.409	3.890	4.085	3.582

4.2 Forecasting Future Brain-Network Dynamics

We next evaluate whether the proposed Forecast TVGL–CFM can predict future brain-network dynamics from an observed prefix of a held-out EEG trial. For each trial TVGL estimates a sequence of precision matrices $\Theta_{1:T}$, and the model observes the first $K = 15$ windows and forecasts the remaining $T - K = 5$ future windows, $p_\theta(\Theta_{K+1:T} | \Theta_{1:K}, y)$.

The training mean alone reaches 2.544 mean AIRM, ahead of persistence at 2.707. Adding the residual flow lowers the error on every dataset and both metrics: 3.287 to 2.748 on BNCI2014_001, 2.847 to 2.415 on Zhou2016, 2.204 to 1.953 on BNCI2015_001, 1.840 to 1.553 on BNCI2014_002, and 2.544 to 2.167 in mean AIRM, a 15% reduction. The gain therefore comes from the learned history-conditioned transport rather than the mean future. All four raw-EEG forecasters decoded through TVGL are beaten on both metrics and all four datasets; the best of them, EEG-GAN, is $1.80\times$ worse in mean AIRM (3.904 against 2.167).

Among models trained on the identical TVGL representation the differences are smaller. TVGL–CFM has the lowest mean AIRM (2.167) and mean logE-RMSE (2.014) and the lowest values on Zhou2016 and BNCI2014_002 on both metrics. DyGraphformer-T, a graph-native dynamic-network forecaster on the same targets, has the lowest AIRM on BNCI2014_001 and BNCI2015_001 and is 0.002 behind on mean AIRM; TVGL–cVAE-Forecast is an additional 0.018 back. TVGL–CFM additionally generates class-conditional trajectories under the same objective.

The two couplings differ substantially on EEG. The mean-centred residual coupling reaches 2.167 mean AIRM and the random-walk coupling 2.592, which is worse than the mean alone. Here the mean is taken per class.

4.3 Chaotic Maps Forecasting

Table 3 shows that the gains extend beyond EEG. Averaged over all nine system $\times$ horizon settings (Lorenz, MacArthur, and Hopfield at $h \in \{3, 5, 7\}$, three seeds each), random walk TVGL–CFM performs best on both metrics, with mean AIRM 1.069 and mean logE-RMSE 1.162. This improves over TVGL–cVAE-Forecast (1.129/1.188), DyGraphformer-T (1.130/1.215), TVGL–DDPM-Forecast (1.137/1.230), persistence (1.607 AIRM), and the best raw-signal pipeline, cWGAN-GP → TVGL (2.138 AIRM). Full results are in Appendix N.

Without class labels, (20) reduces to a global training-set future mean, which reaches AIRM 1.156; the mean-residual flow reaches 1.166, while the history-informed coupling improves to 1.069.

At $h = 5$, TVGL–CFM is best on Hopfield (0.759), TVGL–DDPM-Forecast on Lorenz (1.453 vs. 1.465), and DyGraphformer-T on MacArthur (1.019 vs. 1.047). Overall, TVGL–CFM is the strongest method across the chaotic benchmark.

Table 3: Forecasting results across chaotic systems, averaged over all nine system $\times$ horizon settings (Lorenz, MacArthur and Hopfield at $h \in \{3, 5, 7\}$, fixed history $K = 15$, three seeds each). AIRM is the primary metric and logE-RMSE the secondary; both are lower-is-better. Both TVGL-CFM source constructions are reported. Per-system and per-horizon results are in Appendix N. Best in **bold**, second best *underlined and italicized*.

Method	Mean AIRM	Mean logE-RMSE
TVGL-CFM (Random-Walk)	1.069	1.162
TVGL-CFM (mean-residual)	1.166	1.284
TVGL-cVAE-Forecast	<i>1.129</i>	<i>1.188</i>
TVGL-DDPM-Forecast	1.137	1.230
DyGraphformer-T	1.130	1.215
Training mean (no flow)	1.156	1.208
Mean train displacement	1.607	1.686
Persistence (copy last history)	1.607	1.685
Linear history-drift	2.786	2.889
Basic-CNN $\rightarrow$ TVGL	2.193	2.322
LSTM $\rightarrow$ TVGL	2.018	2.351
cVAE $\rightarrow$ TVGL	4.035	4.165
cWGAN-GP $\rightarrow$ TVGL	2.138	2.269

5 Related work

DIFFEOCFM [45] generates *static* SPD covariance/correlation matrices via pullback-geometry flow matching, while Marti [76] use GANs for static financial correlation matrices. More broadly, deep generative modelling spans VAEs [39], GANs [40], normalizing flows [37, 38], autoregressive models [41] and diffusion [29, 32, 34–36], with intrinsic SPD methods surveyed by Ju et al. [69].

We instead generate and forecast temporal trajectories of inferred graphs. Graph generative models [77–79] primarily target topology, often static and unweighted, rather than temporally coherent SPD precision trajectories. Diffusion forecasters [82] predict future values, whereas we forecast future dependency structure. DyGraphformer [80] infers time-varying dependencies to improve value forecasting; our DyGraphformer-T baseline instead predicts the dependency trajectory itself.

STFlow [83] conditions the source, while CGFM [81] uses an auxiliary forecaster as the source so the flow models its residual structure. Our random-walk coupling follows this principle using parameter-free extrapolation of recent graph-state increments; the mean-centred variant instead keeps the Gaussian source and recentres the target.

6 Discussion and Conclusion

We presented TVGL-CFM, a geometry-aware framework for generating and forecasting time-varying SPD precision trajectories. While the components in our approach have already been established the novelty lies in the coherent combination of them to lift diffeomorphic flow matching from single SPD objects to complete temporal trajectories on the product manifold $(\mathbb{S}_{++}^p)^T$, and learning one globally coupled vector field over the corresponding Td -dimensional Euclidean trajectory space. Although the chart acts window-wise, the learned dynamics do not: each temporal component can depend on the full trajectory through attention. This distinguishes TVGL-CFM from a window-wise application of DIFFEOCFM, which would transport individual SPD matrices without learned temporal communication.

The same trajectory-level construction supports forecasting. We study two data-dependent couplings: a mean-centred residual formulation that recentres the target around a training-set future mean, and a random-walk formulation whose source extrapolates the observed prefix. Across experiments, modelling the precision trajectory directly is substantially stronger than generating raw signals and estimating TVGL afterwards, while remaining competitive with models trained on the same TVGL representation. To our knowledge, this is the first extension of Riemannian diffeomorphic flow matching to time series of matrices.

The main limitations arise from the target estimator and representation. TVGL hyperparameters λ and β constrain the fidelity of the learned targets and are held fixed rather than tuned per dataset, although Appendix G shows no sharp degradation within a factor-of-two neighbourhood of the settings we use.

The embedding dimension also scales quadratically with channel count [44]; Appendix I evaluates the pipeline up to $p = 512$, where dense TVGL and matrix-logarithm evaluation dominate the computational cost. The framework is not tied to TVGL: Appendix H demonstrates transfer to the LOGGLE estimator [9], suggesting broader applicability to temporal graphical-model estimators with compatible geometric structure.

Future work should explore alternative temporal estimators and manifolds, data-dependent couplings such as optimal transport, and sensitivity to temporal choices such as window size.

References

- [1] David Hallac, Youngsuk Park, Stephen Boyd, and Jure Leskovec. Network inference via the time-varying graphical lasso. In *Proc. 23rd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2017.
- [2] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3):432–441, 2008.
- [3] Onureena Banerjee, Laurent El Ghaoui, and Alexandre d’Aspremont. Model selection through sparse maximum likelihood estimation for multivariate Gaussian or binary data. *J. Mach. Learn. Res.*, 9:485–516, 2008.
- [4] Ming Yuan and Yi Lin. Model selection and estimation in the Gaussian graphical model. *Biometrika*, 94(1):19–35, 2007.
- [5] Patrick Danaher, Pei Wang, and Daniela M. Witten. The joint graphical lasso for inverse covariance estimation across multiple classes. *J. R. Stat. Soc. Ser. B*, 76(2):373–397, 2014.
- [6] Karthik Mohan, Palma London, Maryam Fazel, Daniela Witten, and Su-In Lee. Node-based learning of multiple Gaussian graphical models. *J. Mach. Learn. Res.*, 15:445–488, 2014.
- [7] Mladen Kolar, Le Song, Amr Ahmed, and Eric P. Xing. Estimating time-varying networks. *Ann. Appl. Stat.*, 4(1):94–123, 2010.
- [8] Shuheng Zhou, John Lafferty, and Larry Wasserman. Time varying undirected graphs. *Mach. Learn.*, 80(2–3):295–319, 2010.
- [9] Jilei Yang and Jie Peng. Estimating time-varying graphical models. *J. Comput. Graph. Stat.*, 29(1):191–202, 2020.
- [10] Amr Ahmed and Eric P. Xing. Recovering time-varying networks of dependencies in social and biological studies. *Proc. Natl. Acad. Sci. USA*, 106(29):11878–11883, 2009.
- [11] Om Roy, Yashar Moshfeghi, J. Smith, Agustin Ibanez, Mario A. Parra, and Keith M. Smith. A Hodge-FAST framework for high-resolution dynamic functional connectivity analysis of higher-order interactions in EEG signals. In *Proc. 47th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC)*, pages 1–6, 2025. doi: 10.1109/EMBC58623.2025.11253015.
- [12] Ricardo Pio Monti, Peter Hellyer, David Sharp, Robert Leech, Christoforos Anagnostopoulos, and Giovanni Montana. Estimating time-varying brain connectivity networks from functional MRI time series. *NeuroImage*, 103:427–443, 2014.
- [13] David Hallac, Jure Leskovec, and Stephen Boyd. Network lasso: clustering and optimization in large graphs. In *KDD*, 2015.
- [14] Matt Wytock and J. Zico Kolter. Sparse Gaussian conditional random fields: algorithms, theory, and application to energy forecasting. In *ICML*, 2013.
- [15] Steffen L. Lauritzen. *Graphical Models*. Clarendon Press, Oxford, 1996.
- [16] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.
- [17] Håvard Rue and Leonhard Held. *Gaussian Markov Random Fields: Theory and Applications*. CRC Press, 2005.
- [18] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Found. Trends Mach. Learn.*, 3(1):1–122, 2011.
- [19] Neal Parikh and Stephen Boyd. Proximal algorithms. *Found. Trends Optim.*, 1(3):127–239, 2014.
- [20] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

- [21] Yilun Chen, Ami Wiesel, Yonina C. Eldar, and Alfred O. Hero. Shrinkage algorithms for MMSE covariance estimation. *IEEE Trans. Signal Process.*, 58(10):5016–5029, 2010.
- [22] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *ICLR*, 2023.
- [23] Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky T. Q. Chen, David Lopez-Paz, Heli Ben-Hamu, and Itai Gat. Flow matching guide and code. arXiv preprint arXiv:2412.06264, 2024.
- [24] Ricky T. Q. Chen and Yaron Lipman. Flow matching on general geometries. In *ICLR*, 2024.
- [25] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: learning to generate and transfer data with rectified flow. In *ICLR*, 2023.
- [26] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *ICLR*, 2023.
- [27] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Trans. Mach. Learn. Res.*, 2024.
- [28] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.
- [29] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [30] Wang, Y., Ma, Y., Li, W., and You, C. (2026). Let eeg models learn eeg. arXiv preprint arXiv:2605.21280.
- [31] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.
- [32] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, 2019.
- [33] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.
- [34] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *NeurIPS*, 2021.
- [35] Alexander Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021.
- [36] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- [37] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using Real NVP. In *ICLR*, 2017.
- [38] Diederik P. Kingma and Prafulla Dhariwal. Glow: generative flow with invertible 1×1 convolutions. In *NeurIPS*, 2018.
- [39] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *ICLR*, 2014.
- [40] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. In *NeurIPS*, 2014.
- [41] Aaron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *ICML*, 2016.
- [42] Robert J. McCann. A convexity principle for interacting gases. *Adv. Math.*, 128(1):153–179, 1997.
- [43] Cédric Villani. *Optimal Transport: Old and New*. Springer, 2009.
- [44] Kazusato Oko, Shunta Akiyama, and Taiji Suzuki. Diffusion models are minimax optimal distribution estimators. In *ICML*, 2023.
- [45] Antoine Collas, Ce Ju, Nicolas Salvy, and Bertrand Thirion. Riemannian flow matching for brain connectivity matrices via pullback geometry (DIFFEOCFM). In *NeurIPS*, 2025.
- [46] Valentin De Bortoli, Emile Mathieu, Michael Hutchinson, James Thornton, Yee Whye Teh, and Arnaud Doucet. Riemannian score-based generative modelling. In *NeurIPS*, 2022.
- [47] Thibault de Surré, Fabien Lotte, Sylvain Chevallier, and Florian Yger. Wrapped Gaussian on the manifold of symmetric positive definite matrices. In *ICML*, 2025.
- [48] Jaehyeon Jo and Sung Ju Hwang. Generative modeling on manifolds through mixture of Riemannian diffusion processes. arXiv preprint arXiv:2310.07216, 2023.
- [49] Kacper Kapusniak, Peter Potaptchik, Teodora Reu, Leo Zhang, Alexander Tong, Michael Bronstein, Avishek Joey Bose, and Francesco Di Giovanni. Metric flow matching for smooth interpolations on the data manifold. In *NeurIPS*, 2024.

- [50] Benjamin Kurt Miller, Ricky T. Q. Chen, Anuroop Sriram, and Brandon M. Wood. FlowMM: generating materials with Riemannian flow matching. In *ICML*, 2024.
- [51] Haoran Ding, Noémie Jaquier, Jan Peters, and Leonel Rozo. Fast and robust visuomotor Riemannian flow matching policy. *IEEE Trans. Robotics*, 2025.
- [52] Guillaume Huguet, James Vuckovic, Kilian Fatras, Eric Thibodeau-Laufer, Pablo Lemos, Riashat Islam, Cheng-Hao Liu, Jarrod Rector-Brooks, Tara Akhound-Sadegh, Michael M. Bronstein, et al. Sequence-augmented SE(3)-flow matching for conditional protein generation. In *NeurIPS*, 2024.
- [53] Luca Falorsi, Pim de Haan, Tim R. Davidson, and Patrick Forré. Reparameterizing distributions on Lie groups. In *AISTATS*, 2019.
- [54] Yunchen Li, Zhou Yu, Gaoqi He, Yunhang Shen, Ke Li, Xing Sun, and Shaohui Lin. SPD-DDPM: denoising diffusion probabilistic models in the symmetric positive definite space. In *AAAI*, 2024.
- [55] Vincent Arsigny, Pierre Fillard, Xavier Pennec, and Nicholas Ayache. Geometric means in a novel vector space structure on symmetric positive-definite matrices. *SIAM J. Matrix Anal. Appl.*, 29(1):328–347, 2007.
- [56] Xavier Pennec, Pierre Fillard, and Nicholas Ayache. A Riemannian framework for tensor computing. *Int. J. Comput. Vis.*, 66(1):41–66, 2006.
- [57] Xavier Pennec. Intrinsic statistics on Riemannian manifolds: basic tools for geometric measurements. *J. Math. Imaging Vis.*, 25(1):127–154, 2006.
- [58] Zhenhua Lin. Riemannian geometry of symmetric positive definite matrices via Cholesky decomposition. *SIAM J. Matrix Anal. Appl.*, 40(4):1353–1370, 2019.
- [59] Lene Theil Skovgaard. A Riemannian geometry of the multivariate normal model. *Scand. J. Stat.*, 11(4):211–223, 1984.
- [60] Om Roy, Yashar Moshfeghi, and Keith M. Smith. Covariance density neural networks. *Trans. Mach. Learn. Res.*, 2026.
- [61] Yann Thanwerdas. Riemannian and stratified geometries on covariance and correlation matrices. PhD thesis, Université Côte d’Azur, 2022.
- [62] Paul David and Weiqing Gu. A Riemannian structure for correlation matrices. *Oper. Matrices*, 13(3):607–627, 2019.
- [63] P. Thomas Fletcher and Sarang Joshi. Principal geodesic analysis on symmetric spaces: statistics of diffusion tensors. In *MICCAI Workshop (CVAMIA)*, 2004.
- [64] Alexandre Barachant, Stéphane Bonnet, Marco Congedo, and Christian Jutten. Multiclass brain–computer interface classification by Riemannian geometry. *IEEE Trans. Biomed. Eng.*, 59(4):920–928, 2012.
- [65] Benjamin Blankertz, Ryota Tomioka, Steven Lemm, Motoaki Kawanabe, and Klaus-Robert Müller. Optimizing spatial filters for robust EEG single-trial analysis. *IEEE Signal Process. Mag.*, 25(1):41–56, 2008.
- [66] Fabien Lotte, Laurent Bougrain, Andrzej Cichocki, Maureen Clerc, Marco Congedo, Alain Rakotomamonjy, and Florian Yger. A review of classification algorithms for EEG-based brain–computer interfaces: a 10 year update. *J. Neural Eng.*, 15(3):031005, 2018.
- [67] Gert Pfurtscheller and Fernando H. Lopes da Silva. Event-related EEG/MEG synchronization and desynchronization: basic principles. *Clin. Neurophysiol.*, 110(11):1842–1857, 1999.
- [68] Reinmar J. Kobler, Jun-ichiro Hirayama, Qibin Zhao, and Motoaki Kawanabe. SPD domain-specific batch normalization to crack interpretable unsupervised domain adaptation in EEG. In *NeurIPS*, 2022.
- [69] Ce Ju, Reinmar J. Kobler, Antoine Collas, Motoaki Kawanabe, Cuntai Guan, and Bertrand Thirion. SPD learning for covariance-based neuroimaging analysis: perspectives, methods, and challenges. arXiv preprint arXiv:2504.18882, 2025.
- [70] Vinay Jayaram and Alexandre Barachant. MOABB: trustworthy algorithm benchmarking for BCIs. *J. Neural Eng.*, 15(6):066011, 2018.
- [71] Michael D. Fox, Randy L. Buckner, Hesheng Liu, M. Mallar Chakravarty, Andres M. Lozano, and Alvaro Pascual-Leone. Resting-state networks link invasive and noninvasive brain stimulation across diverse psychiatric and neurological diseases. *Proc. Natl. Acad. Sci. USA*, 111(41):E4367–E4375, 2014.
- [72] Emily L. Dennis and Paul M. Thompson. Functional brain connectivity using fMRI in aging and Alzheimer’s disease. *Neuropsychol. Rev.*, 24(1):49–62, 2014.
- [73] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [74] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high-frequency functions in low-dimensional domains. In *NeurIPS*, 2020.

- [75] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [76] Gautier Marti. CorrGAN: sampling realistic financial correlation matrices using generative adversarial networks. In *ICASSP*, 2020.
- [77] Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. DiGress: discrete denoising diffusion for graph generation. In *ICLR*, 2023.
- [78] Xiaoyang Huang, Tao Ruan, et al. Graph generation with spectral geodesic flow matching. arXiv preprint arXiv:2510.02520, 2025.
- [79] R. Williams et al. Scalable generative modeling of weighted graphs. arXiv preprint arXiv:2507.23111, 2025.
- [80] Shuo Han, Yaling Xun, Jianghui Cai, Haifeng Yang, and Yanfeng Li. DyGraphformer: transformer combining dynamic spatio-temporal graph network for multivariate time series forecasting. *Neural Networks*, 181:106776, 2025.
- [81] Huibo Xu, Runlong Yu, Likang Wu, Xianquan Wang, and Qi Liu. Bridging the last mile of prediction: enhancing time series forecasting with conditional guided flow matching. *arXiv preprint arXiv:2507.07192*, 2025.
- [82] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting (TimeGrad). In *ICML*, 2021.
- [83] Koen Bennema ten Brinke, Koen Minartz, and Vlado Menkovski. STFlow: data-coupled flow matching for geometric trajectory simulation. In *ICML*, 2026.
- [84] Suman Ravuri and Oriol Vinyals. Classification accuracy score for conditional generative models. In *NeurIPS*, 2019.

A Extended background

A.1 Riemannian structures on SPD and correlation matrices

The log-Euclidean metric of (1) is one of several Riemannian structures proposed for SPD and correlation matrices, including the affine-invariant metric [57, 59], the Cholesky-based geometries [58], and the stratified constructions of Thanwerdas [61] and David and Gu [62]; it is the choice under which flow matching reduces to a Euclidean problem. Such geometries also underpin classical manifold statistics such as principal geodesic analysis [63]. Other global charts on $\mathbb{S}_{++}^p$, such as the log-Cholesky map [58], could be substituted without changing the framework or any of the results of Appendix E.

A.2 Generative modelling on manifolds

Riemannian flow matching [24] sits within a broader effort to lift generative models onto manifolds, including Riemannian score-based diffusion [46], mixtures of Riemannian diffusion processes [48], data-manifold metric flow matching [49], and applications to materials [50], proteins [52], robot policies [51], and Lie-group densities [53]. For SPD data specifically, SPD-DDPM [54] and wrapped Gaussians [47] operate intrinsically but at higher cost. Minibatch optimal-transport couplings [27] can further straighten the conditional paths of (3). A practical overview of the objective and its variants is given by Lipman et al. [23].

A.3 Data-dependent couplings

Standard flow matching pairs each data sample with an independent draw from a simple source distribution. Choosing this pairing more carefully, for instance through optimal-transport couplings [27], which are grounded in displacement interpolation [42, 43], can shorten transport and speed up training. In conditional forecasting, however, the observed history already contains information about the future. Recent trajectory-simulation work exploits this by constructing a source sample from the observed prefix, for example a noisy continuation of the recent motion, so that the flow transports from an informed prior rather than from uninformative Gaussian noise [83]. A closely related idea in time-series forecasting is CGFM [81], which uses the predictive distribution of an auxiliary forecaster as the source, so that flow matching learns the probabilistic structure of that

forecaster’s residuals rather than treating them purely as an optimisation target. Both modify or condition the *source*, and our random-walk coupling of Section 3.4 follows that principle, with the state a log-Euclidean precision graph rather than a particle position and the source a parameter-free stochastic extrapolation of recent graph-state increments in place of a separately trained auxiliary forecaster. Our mean-centred residual coupling is a different construction: the source stays Gaussian and the target is recentred by a training-set future mean. Since that mean has no learned parameters, running it forward with a zero residual gives the zero-residual baseline reported alongside the full model.

A.4 Time-varying network inference

TVGL [1] has been applied to domains as different as neuroscience, finance, and automotive sensing. Its precursors on time-varying network inference include kernel smoothing [8], ℓ_1 -fused penalties [7], and applications to biological and brain networks [10, 12]; the underlying conditional-independence semantics are standard in the graphical-model literature [15–17], and closely related structured estimators appear in Banerjee et al. [3], Yuan and Lin [4], Hallac et al. [13], Wytock and Kolter [14].

A.5 Motor-imagery decoding

Covariance-based Riemannian representations and data-driven graph constructions provide established ways to characterise multiscale and higher-order structure in EEG signals [11, 60, 64, 68]. In motor imagery, discriminative information is largely expressed through sensorimotor-rhythm (de)synchronisation [67], commonly exploited by spatial filters such as CSP [65, 66].

B Workflow overview and the two time variables

The generative model, and its two time variables. The generator is a *single transformer* that reads a whole precision trajectory as a sequence of tokens (one token per window) and outputs the flow’s velocity field; it is defined in full in (9) of Section 3.2. It is entirely separate from the TVGL stage: TVGL is a convex optimisation that produces the target trajectories, whereas the transformer is the neural model trained on those targets. This model reads two distinct notions of time. *Flow time* $s \in [0, 1]$ is the CFM transport variable: $s = 0$ is the source, $s = 1$ is the target, and sampling integrates the ODE in s . *Window time* $\tau_i := i/(T - 1)$ indexes the fixed TVGL windows $i = 1, \dots, T$ and tells the transformer where each token lies along the trajectory. The model receives both through separate Fourier features FF_s and FF_τ (see (9)). When needed, we write $z_{s,i}$ for flow time s at window i ; in particular, $\hat{z}_{1,i}$ is the predicted clean endpoint at window i .

Figure 2 summarises the two tasks that share the TVGL target-construction and log-Euclidean embedding stages of Section 3: class-conditional generation of a whole precision trajectory, and history-conditioned forecasting of its continuation.

C TVGL target construction and solver

This appendix expands Section 3.1 of the main paper: the shrinkage covariance estimator, the batched ADMM solver for (4), and the relationship between SPD validity and sparsity in the decoded output.

C.1 Windowed shrinkage covariance

Each sequence’s p -channel signal is z -scored per channel over time and split into T contiguous windows of length n . Because n may be comparable to p , the raw sample covariance is poorly conditioned; we use the oracle-approximating-shrinkage (OAS) estimator [21], which for a window with sample covariance S returns

$$\hat{S} = (1 - \rho) S + \rho \hat{\mu} I, \quad \hat{\mu} = \frac{1}{p} \text{tr} S, \quad \rho = \min\left(1, \frac{\alpha + \hat{\mu}^2}{(n + 1)(\alpha - \hat{\mu}^2/p)}\right), \quad \alpha = \frac{1}{p^2} \text{tr}(S^2). \quad (24)$$

All windows are computed in a single vectorised pass over (sequence, window). We divide every $\hat{S}$ by the mean training diagonal so that covariances are $O(1)$ and the TVGL λ is interpretable.

C.2 Batched TVGL targets

We solve (4) for all sequences and windows simultaneously with ADMM [18, 20]. Introducing consensus copies Z_0, Z_1, Z_2 for the sparsity and the two temporal roles of each slice, the updates have

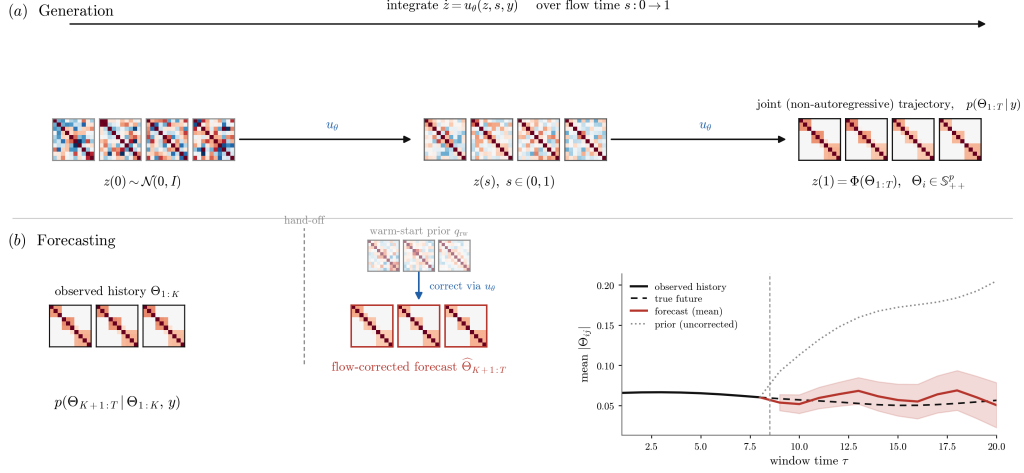


Figure 2: TVGL-CFM generation and forecasting workflows. (a) Generation. A class-conditioned conditional flow matching model transports a simple source trajectory $z(0) \sim \mathcal{N}(0, I)$ to a joint TVGL precision trajectory by integrating the learned vector field $\dot{z} = u_\theta(z, s, y)$ over flow time $s \in [0, 1]$. The terminal state is decoded as $z(1) = \Phi(\Theta_{1:T})$, yielding a non-autoregressive sample from the conditional trajectory distribution $p(\Theta_{1:T} | y)$, where each $\Theta_i \in \mathbb{S}_{++}^p$. **(b) Forecasting.** Given an observed TVGL history $\Theta_{1:K}$, the model integrates the learned vector field under one of two data-dependent couplings, either Gaussian residuals around the training-mean future m^f or a random-walk random-walk source, and decodes to obtain a forecast $\hat{\Theta}_{K+1:T}$. The right panel illustrates the resulting dynamic forecast summary: observed history, true future, source trajectory, and the TVGL-CFM predictive mean with uncertainty over the forecast horizon. The vertical dashed line marks the handoff between observed and predicted windows.

closed forms in terms of proximal operators [19]. Consider a single window (we suppress the window index i). The likelihood (proximal) update first forms the symmetric matrix $M = \frac{1}{2}(A + A^\top) - \eta S$, where A is the averaged combination of the consensus and dual variables for that slice (Algorithm 1, line 3), S is its shrinkage covariance, and $\eta = n/(\rho c_i) > 0$ is the per-slice weight defined on line 1 (n samples per window, penalty parameter ρ , and c_i the number of active roles of slice i). Taking the eigendecomposition $M = Q \text{diag}(d) Q^\top$, with orthogonal eigenvector matrix Q and eigenvalue vector $d \in \mathbb{R}^p$, the update has the closed form

$$\Theta = Q \text{diag}\left(\frac{1}{2}(d + \sqrt{d^2 + 4\eta})\right) Q^\top \succ 0, \quad (25)$$

which is guaranteed SPD because every transformed eigenvalue $\frac{1}{2}(d_j + \sqrt{d_j^2 + 4\eta})$ is strictly positive. The sparsity update is the off-diagonal soft-threshold $\mathcal{S}_{\lambda/\rho}$ (diagonal preserved). The temporal update is the proximal operator of the same evolutionary penalty ψ chosen in (4) (Section 2.3), applied to the consensus difference; for the three choices of ψ it is

$$\text{prox}_{\kappa\psi}(D) = \begin{cases} \mathcal{S}_\kappa(D), & \psi = \ell_1 \text{ (elementwise)}, \\ D/(1 + 2\kappa), & \psi = \ell_2^2 \text{ (Laplacian)}, \\ \left[\max(0, 1 - \kappa/\|[D]_j\|_2) [D]_j \right]_j, & \psi = \text{group } \ell_2 \text{ (per column } j). \end{cases} \quad (26)$$

Algorithm 1 states the procedure; the output $\Theta_{n,i}$ is a sparse SPD precision trajectory per sequence.

Algorithm 1 Batched TVGL via ADMM (all sequences/windows)

Require: shrinkage covariances $S_{n,i}$; λ, β, ρ ; samples/window n ; iterations K_{admm}

- 1: init $\Theta, Z_0, Z_1, Z_2 \leftarrow I$; $U_0, U_1, U_2 \leftarrow 0$; $c_i \leftarrow 1 + [i < T] + [i > 0]$; $\eta_i \leftarrow n/(\rho c_i)$
- 2: **for** $k = 1$ to K_{admm} **do**
- 3: $A_i \leftarrow (Z_{0,i} - U_{0,i} + [i < T](Z_{1,i} - U_{1,i}) + [i > 1](Z_{2,i} - U_{2,i}))/c_i$
- 4: $\Theta_i \leftarrow \text{Eq. (25) with } M_i = \frac{1}{2}(A_i + A_i^\top) - \eta_i S_i$ ▷ batched eigendecomposition
- 5: $Z_{0,i} \leftarrow \mathcal{S}_{\lambda/\rho}(\Theta_i + U_{0,i})$ off-diagonal; diagonal $\leftarrow (\Theta_i + U_{0,i})$
- 6: $E_i \leftarrow \text{prox}_{(2\beta/\rho)\psi}((\Theta_i + U_{2,i}) - (\Theta_{i-1} + U_{1,i-1}))$ ▷ Eq. (26)
- 7: $Z_{1,i-1} \leftarrow \frac{1}{2}(\Sigma_i - E_i)$, $Z_{2,i} \leftarrow \frac{1}{2}(\Sigma_i + E_i)$ with $\Sigma_i = (\Theta_{i-1} + U_{1,i-1}) + (\Theta_i + U_{2,i})$
- 8: $U_0 += \Theta - Z_0$; $U_1 += \Theta - Z_1$; $U_2 += \Theta - Z_2$ ▷ on active slices
- 9: **end for**
- 10: **return** $\frac{1}{2}(\Theta + \Theta^\top)$

C.3 SPD validity and exact sparse readout

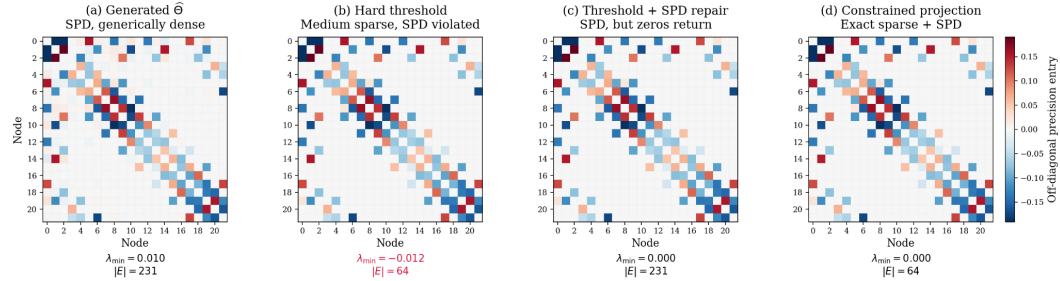


Figure 3: Illustration of SPD validity versus exact sparsity for a TVGL-CFM generated precision matrix. The decoded output is symmetric positive-definite by construction but generically dense. Hard thresholding yields an exact sparse support but can violate positive-definiteness. A generic eigenvalue-based SPD repair restores positive-definiteness but reintroduces nonzero entries on the thresholded support complement. The constrained projection instead computes the nearest matrix satisfying both the prescribed zero pattern and $\Theta \succeq \varepsilon I$, yielding an exactly sparse SPD readout.

Decoding through ϕ^{-1} guarantees that every generated $\hat{\Theta}_i$ is symmetric positive-definite, since the inverse log-Euclidean map is the matrix exponential. This guarantees manifold validity, but not exact sparsity: the exponential of a generic symmetric matrix is dense, so entries that are structurally zero in the sparse TVGL targets are typically represented by small but nonzero values in the generated precision matrices.

We therefore distinguish between the sparse TVGL targets used for training and oracle evaluation, the dense SPD trajectories produced directly by the model, and an optional sparse SPD readout. All quantitative results are computed on the direct model output, without post-processing. The sparse readout is intended for applications where an exact graph support is required.

A naive threshold-and-project procedure is insufficient for this purpose. Thresholding small off-diagonal entries can destroy positive-definiteness, while a subsequent unconstrained projection back onto $\mathbb{S}_{++}^p$ can reintroduce nonzero values at entries that were explicitly thresholded to zero. Instead, we first extract a candidate support from the generated precision matrix. For a threshold $\tau > 0$, define

$$\mathcal{S}_\tau(\hat{\Theta}_i) = \left\{ (a, b) : a = b \text{ or } |\hat{\Theta}_{i,ab}| \geq \tau \right\}. \quad (27)$$

We then compute the nearest matrix that simultaneously respects this support and remains strictly positive-definite:

$$\Theta_i^{\text{sp}} = \arg \min_{\Theta \in \mathbb{S}^p} \frac{1}{2} \left\| \Theta - \hat{\Theta}_i \right\|_F^2 \quad (28)$$

subject to

$$\Theta \succeq \varepsilon I, \quad \Theta_{ab} = 0 \quad \forall (a, b) \notin \mathcal{S}_\tau(\hat{\Theta}_i), \quad (29)$$

where $\varepsilon > 0$ is a small eigenvalue margin.

This is a convex optimization problem. Importantly, the retained nonzero entries are not frozen to their values in $\hat{\Theta}_i$; they may move slightly so that positive-definiteness can be restored while remaining as close as possible to the generated matrix. Consequently, the solution satisfies both

$$\lambda_{\min}(\Theta_i^{\text{sp}}) \geq \varepsilon \quad (30)$$

and

$$\Theta_{i,ab}^{\text{sp}} = 0 \quad \forall (a,b) \notin \mathcal{S}_\tau(\hat{\Theta}_i), \quad (31)$$

so the resulting readout is simultaneously SPD and exactly sparse.

For consistency with the TVGL estimator, we may choose $\tau = \lambda/\rho$, matching the cutoff appearing in the TVGL sparsity proximal update. Alternatively, τ can be treated as a visualization or support-selection parameter. This operation is applied only after generation and does not alter the learned flow or the reported quantitative metrics. Thus, the model output itself remains the dense SPD trajectory defined by the manifold decoder, while the constrained projection provides an optional exact sparse graph representation when one is required.

D Training and sampling algorithms

Algorithms 2 and 3 give training and sampling for the class-conditional generator of Section 3.2, and Algorithm 4 gives block, non-autoregressive forecasting under either coupling of Section 3.4.

Algorithm 2 TVGL–CFM generation training

Require: standardised target trajectories z_n , labels y_n ; window-time grid $\tau_{1:T}$, $\tau_i = i/(T-1)$;
weight λ_{temp}

- 1: **while** not converged **do**
- 2: sample batch (z_1, y) ; $z_0 \sim \mathcal{N}(0, I)$; flow time $s \sim \mathcal{U}[0, 1]$
- 3: $z_s \leftarrow (1-s)z_0 + s z_1$; $v \leftarrow u_\theta(z_s, s, y, \tau_{1:T})$; $\hat{z}_1 \leftarrow z_s + (1-s)v$
- 4: $\mathcal{L} \leftarrow \|v - (z_1 - z_0)\|_2^2 + \lambda_{\text{temp}} \frac{1}{T-1} \sum_{i=2}^T \psi_E(\hat{z}_{1,i} - \hat{z}_{1,i-1})$
- 5: update θ by AdamW on $\mathcal{L}$
- 6: **end while**

Algorithm 3 TVGL–CFM generation sampling

Require: label y ; flow-time steps L ; window-time grid $\tau_{1:T}$; standardiser (μ, σ)

- 1: $z \sim \mathcal{N}(0, I)$; $h \leftarrow 1/L$ ▷ flow-time step
- 2: **for** $\ell = 0$ to $L-1$ **do** ▷ integrate ODE in flow time at fixed $\tau_{1:T}$
- 3: $z \leftarrow \text{RK4-step}(u_\theta(\cdot, \cdot, y, \tau_{1:T}), z, \ell h, h)$
- 4: **end for**
- 5: $z \leftarrow \sigma \odot z + \mu$; $\hat{\Theta} \leftarrow \phi^{-1}(z)$ ▷ matrix exponential per window
- 6: **optional** (for ℓ_1): off-diagonal soft-threshold $\hat{\Theta}$, then project to $\mathbb{S}_{++}^p$
- 7: **return** $\hat{\Theta}$

Algorithm 4 TVGL-CFM forecasting under either data-dependent coupling (block, non-autoregressive)

Require: history $z_{1:K}$, optional label y ; coupling $\in \{\text{mean-residual, random-walk}\}$; training mean m^f (per class where labels exist) from Eq. (20); flow-time steps L ; ensemble size M ; window-time grid $\tau_{K+1:T}$; standardiser (μ, σ)

- 1: $H \leftarrow \mathcal{E}_\theta(z_{1:K}) \in \mathbb{R}^{K \times d_{\text{model}}}$ $\triangleright$ all K history tokens, Eq. (15)
- 2: **if** coupling = random-walk **then**
- 3: estimate $\hat{\mu}, \hat{\sigma}$ from recent history increments by Eq. (22)
- 4: **end if**
- 5: **for** $m = 1$ to M **do**
- 6: **if** coupling = mean-residual **then**
- 7: $z^f \sim \mathcal{N}(0, I) \in E_f$ $\triangleright$ residual coordinates
- 8: **else**
- 9: $z^f \sim p_{\text{RW}}(\cdot \mid z_{1:K})$ by Eq. (23) $\triangleright$ absolute coordinates
- 10: **end if**
- 11: $h \leftarrow 1/L$
- 12: **for** $\ell = 0$ to $L - 1$ **do** $\triangleright$ integrate ODE in flow time at fixed $\tau_{K+1:T}$
- 13: $z^f \leftarrow \text{RK4-step}(u_\theta(\cdot, \cdot, y, \tau_{K+1:T}, H), z^f, \ell h, h)$
- 14: **end for**
- 15: $z \leftarrow m^f + z^f$ if mean-residual, else $z \leftarrow z^f$
- 16: $\hat{\Theta}_{K+1:T}^{(m)} \leftarrow \phi^{-1}(\sigma \odot z + \mu)$
- 17: **end for**
- 18: **return** ensemble $\{\hat{\Theta}_{K+1:T}^{(m)}\}_{m=1}^M$ (point forecast: mean in embedding; spread: uncertainty)

E Pullback Geometry for Time-Varying Precision Trajectories

This appendix lifts the pullback-CFM framework of Collas et al. [45] from a single SPD matrix to a trajectory of SPD precision matrices, adds embedding standardisation, and establishes the equivalences on which the generation and forecasting models of Section 3 both rely, including the data-dependent source couplings of Section 3.4. All results are stated for the log-Euclidean diffeomorphism of Eq. (1), but extend verbatim to any global diffeomorphism $\mathbb{S}_{++}^p \rightarrow \mathbb{R}^d$ (e.g. log-Cholesky).

E.1 Setup and notation

Let p be the channel count, $d := p(p+1)/2$, and T the trajectory length. Recall the log-Euclidean diffeomorphism from Eq. (1):

$$\phi : \mathbb{S}_{++}^p \rightarrow \mathbb{R}^d, \quad \phi(\Theta) = \text{vec}_{\text{lt}}(\log \Theta), \quad \phi^{-1} : \mathbb{R}^d \rightarrow \mathbb{S}_{++}^p, \quad \phi^{-1}(\eta) = \exp(\text{vec}_{\text{lt}}^{-1}(\eta)), \quad (32)$$

where vec_{lt} vectorises the lower-triangular part with off-diagonal entries scaled by $\sqrt{2}$ so that

$$\|\phi(\Theta) - \phi(\Theta')\|_2 = \|\log \Theta - \log \Theta'\|_F = d_{\text{LE}}(\Theta, \Theta'), \quad (33)$$

i.e. vec_{lt} is an isometry between the symmetric matrices $(\mathbb{S}^p, \langle \cdot, \cdot \rangle_F)$ and $(\mathbb{R}^d, \langle \cdot, \cdot \rangle_2)$.

Product manifold. The trajectory manifold is

$$\mathcal{M}_T := \underbrace{\mathbb{S}_{++}^p \times \cdots \times \mathbb{S}_{++}^p}_{T \text{ factors}}, \quad (34)$$

with tangent space $T_{\Theta_{1:T}} \mathcal{M}_T = \prod_{i=1}^T T_{\Theta_i} \mathbb{S}_{++}^p$. We equip $\mathcal{M}_T$ with the *product log-Euclidean metric*

$$\langle \xi, \eta \rangle_{\Theta_{1:T}}^{\text{prod}} := \sum_{i=1}^T \langle \xi_i, \eta_i \rangle_{\Theta_i}^{\text{LE}}. \quad (35)$$

Let $E := \mathbb{R}^{T \times d}$ with the standard Euclidean inner product, and define

$$\Phi : \mathcal{M}_T \rightarrow E, \quad \Phi(\Theta_{1:T}) := (\phi(\Theta_1), \dots, \phi(\Theta_T)). \quad (36)$$

Identities used throughout. Two identities of differentials, which follow from the chain rule applied to $\Phi \circ \Phi^{-1} = \text{Id}_E$ and $\Phi^{-1} \circ \Phi = \text{Id}_{\mathcal{M}_T}$, are

$$D\Phi(\Phi^{-1}(z)) \circ D\Phi^{-1}(z) = \text{Id}_E, \quad D\Phi^{-1}(\Phi(x)) \circ D\Phi(x) = \text{Id}_{T_x \mathcal{M}_T}. \quad (37)$$

E.2 Proof of Proposition 1 (product diffeomorphism and pullback metric)

Proof of Proposition 1. Diffeomorphism. ϕ is a C^∞ bijection $\mathbb{S}_{++}^p \rightarrow \mathbb{R}^d$ with C^∞ inverse, since the matrix logarithm and exponential are smooth on $\mathbb{S}_{++}^p$ and on the space $\mathbb{S}^p$ of symmetric matrices respectively, and vec_{lt} is a linear isomorphism. The map

$$\Phi = \phi \times \cdots \times \phi : (\mathbb{S}_{++}^p)^T \longrightarrow (\mathbb{R}^d)^T \cong \mathbb{R}^{T \times d}$$

is therefore a C^∞ bijection with C^∞ inverse $\Phi^{-1} = \phi^{-1} \times \cdots \times \phi^{-1}$, hence a global diffeomorphism.

Pullback metric. The differential $D\Phi(\Theta_{1:T})$ is block-diagonal in the trajectory index: for $\xi \in T_{\Theta_{1:T}} \mathcal{M}_T$,

$$D\Phi(\Theta_{1:T})[\xi] = (D\phi(\Theta_1)[\xi_1], \dots, D\phi(\Theta_T)[\xi_T]) \in E. \quad (38)$$

Writing $g_E(u, v) = \sum_{i,k} u_{i,k} v_{i,k} = \sum_i \langle u_i, v_i \rangle_2$ and applying (38) to $\xi, \eta \in T_{\Theta_{1:T}} \mathcal{M}_T$,

$$\begin{aligned} (\Phi^* g_E)_{\Theta_{1:T}}(\xi, \eta) &= g_E(D\Phi(\Theta_{1:T})[\xi], D\Phi(\Theta_{1:T})[\eta]) \\ &= \sum_{i=1}^T \langle D\phi(\Theta_i)[\xi_i], D\phi(\Theta_i)[\eta_i] \rangle_2 \\ &= \sum_{i=1}^T (\phi^* g_E)_{\Theta_i}(\xi_i, \eta_i) = \sum_{i=1}^T \langle \xi_i, \eta_i \rangle_{\Theta_i}^{\text{LE}}, \end{aligned}$$

where the third equality is the definition of the pullback metric on each factor and the fourth uses that $\phi^* g_E$ is the log-Euclidean metric on $\mathbb{S}_{++}^p$ [45, Sec. 3.2]. The right-hand side is the product log-Euclidean metric (35). $\square$

E.3 Proof of Proposition 3 (trajectory CFM loss equivalence)

Proposition 3 is stated in Section 3.2; the pullback field (10) and target loss (11) are as defined there.

Proof of Proposition 3. Fix a label y and draw $\Theta_{1:T}^0 \sim p(\cdot | y)$, $\Theta_{1:T}^1 \sim q(\cdot | y)$. Set $z_0 := \Phi(\Theta_{1:T}^0)$, $z_1 := \Phi(\Theta_{1:T}^1)$, and for $s \in [0, 1]$,

$$z_s := (1-s)z_0 + s z_1 \in E, \quad \gamma(s) := \Phi^{-1}(z_s) \in \mathcal{M}_T. \quad (39)$$

γ is the geodesic on $(\mathcal{M}_T, \Phi^* g_E)$. On each factor, $\gamma_i(s) = \phi^{-1}((1-s)\phi(\Theta_i^0) + s\phi(\Theta_i^1))$ is the log-Euclidean geodesic from Θ_i^0 to Θ_i^1 (45, Eq. 14 applied to ϕ). Geodesics on a product Riemannian manifold equipped with the product metric decouple into component-wise geodesics, so γ is the geodesic on $(\mathcal{M}_T, \Phi^* g_E)$ connecting $\Theta_{1:T}^0$ to $\Theta_{1:T}^1$.

Velocity. The straight line z_s in E has constant derivative $\dot{z}_s = z_1 - z_0$. Applying the chain rule to $\gamma = \Phi^{-1} \circ z$,

$$\dot{\gamma}(s) = D\Phi^{-1}(z_s)[\dot{z}_s] = D\Phi^{-1}(z_s)[z_1 - z_0]. \quad (40)$$

At every s , the pullback metric satisfies

$$\|\zeta\|_{\gamma(s)}^2 = (\Phi^* g_E)_{\gamma(s)}(\zeta, \zeta) = \|D\Phi(\gamma(s))[\zeta]\|_E^2 \quad \text{for all } \zeta \in T_{\gamma(s)} \mathcal{M}_T. \quad (41)$$

Set $\zeta := u_{\theta}^{\mathcal{M}_T}(s, \gamma(s), y) - \dot{\gamma}(s)$. By linearity of the differential,

$$D\Phi(\gamma(s))[\zeta] = D\Phi(\gamma(s))[u_{\theta}^{\mathcal{M}_T}(s, \gamma(s), y)] - D\Phi(\gamma(s))[\dot{\gamma}(s)].$$

The first term is $u_{\theta}^E(s, z_s, y)$ by (10) (using $\gamma(s) = \Phi^{-1}(z_s)$). For the second term, (40) gives $D\Phi(\gamma(s))[\dot{\gamma}(s)] = D\Phi(\gamma(s)) \circ D\Phi^{-1}(z_s)[z_1 - z_0] = z_1 - z_0$, where the last equality is the left identity in (37). Combining,

$$D\Phi(\gamma(s))[u_{\theta}^{\mathcal{M}_T}(s, \gamma(s), y) - \dot{\gamma}(s)] = u_{\theta}^E(s, z_s, y) - (z_1 - z_0). \quad (42)$$

Substituting into (41):

$$\|u_{\theta}^{\mathcal{M}_T}(s, \gamma(s), y) - \dot{\gamma}(s)\|_{\gamma(s)}^2 = \|u_{\theta}^E(s, z_s, y) - (z_1 - z_0)\|_E^2. \quad (43)$$

Expectation. (43) holds pointwise for every realisation of $(s, y, \Theta_{1:T}^0, \Theta_{1:T}^1)$. Taking expectation under $s \sim \mathcal{U}[0, 1]$, $y \sim \pi_Y$, $\Theta_{1:T}^0 \sim p(\cdot | y)$, $\Theta_{1:T}^1 \sim q(\cdot | y)$, and using the substitution $z_0 = \Phi(\Theta_{1:T}^0)$, $z_1 = \Phi(\Theta_{1:T}^1)$ (whose marginal laws are $\Phi_{\#}p(\cdot | y)$ and $\Phi_{\#}q(\cdot | y)$ respectively) yields (11). $\square$

E.4 Proof of Proposition 4 (trajectory ODE equivalence)

Proof of Proposition 4. Time derivative. By the chain rule applied to $\Theta_{1:T} = \Phi^{-1} \circ z$,

$$\dot{\Theta}_{1:T}(s) = D\Phi^{-1}(z(s))[\dot{z}(s)] = D\Phi^{-1}(z(s))[u_{\theta}^E(s, z(s), y)],$$

where the second equality uses the ODE for z . Substituting $z(s) = \Phi(\Theta_{1:T}(s))$ and using the definition (10),

$$u_{\theta}^E(s, \Phi(\Theta_{1:T}(s)), y) = D\Phi(\Theta_{1:T}(s))[u_{\theta}^{\mathcal{M}_T}(s, \Theta_{1:T}(s), y)].$$

Therefore

$$\dot{\Theta}_{1:T}(s) = D\Phi^{-1}(\Phi(\Theta_{1:T}(s))) \circ D\Phi(\Theta_{1:T}(s))[u_{\theta}^{\mathcal{M}_T}(s, \Theta_{1:T}(s), y)] = u_{\theta}^{\mathcal{M}_T}(s, \Theta_{1:T}(s), y),$$

by the right identity in (37).

Initial condition. $\Theta_{1:T}(0) = \Phi^{-1}(z(0)) = \Phi^{-1}(\Phi(\Theta_{1:T}^0)) = \Theta_{1:T}^0$.

Slice-wise SPD. For each i , $\Theta_i(s) = \phi^{-1}(z_i(s)) = \exp(\text{vec}_{\text{lt}}^{-1}(z_i(s)))$. Since $\text{vec}_{\text{lt}}^{-1} : \mathbb{R}^d \rightarrow \mathbb{S}^p$ is a linear isomorphism onto the symmetric matrices, $\text{vec}_{\text{lt}}^{-1}(z_i(s)) \in \mathbb{S}^p$. The matrix exponential restricts to a bijection $\exp : \mathbb{S}^p \rightarrow \mathbb{S}_{++}^p$ (diagonalise: $S = Q\Lambda Q^{\top}$ implies $\exp(S) = Q\exp(\Lambda)Q^{\top}$ with positive eigenvalues), so $\Theta_i(s) \in \mathbb{S}_{++}^p$ for every i, s . $\square$

E.5 Proof of Proposition 5 (trajectory Runge–Kutta equivalence)

Proposition 5 is stated in Section 3.2; the scheme has ν stages with Butcher tableau (a_{ij}, b_i, c_i) and step size h .

Proof of Proposition 5. Block-diagonal structure of all manifold operations. On $(\mathcal{M}_T, \Phi^*g_E)$ all the geometric operations used by an explicit Riemannian Runge–Kutta scheme, namely the differential $D\Phi$ (already established as block-diagonal in (38)), its inverse $D\Phi^{-1}$, the exponential map $\text{Exp}_{\Theta_{1:T}}(\zeta) = \Phi^{-1}(\Phi(\Theta_{1:T}) + D\Phi(\Theta_{1:T})[\zeta])$, and parallel transport $\text{PT}_{x \rightarrow y}(\eta) = (D\Phi(y))^{-1}(D\Phi(x)[\eta])$, all decompose into the corresponding operations on each factor $\mathbb{S}_{++}^p$. This is because the product metric is the sum of factor-wise metrics, the diffeomorphism is the Cartesian product of factor diffeomorphisms, and the exponential/parallel-transport formulas [45, Eqs. 15–16] are determined pointwise by $D\Phi$.

Reduction to the single-matrix case. The Riemannian Runge–Kutta scheme on $\mathcal{M}_T$ at step ℓ evaluates $u_{\theta}^{\mathcal{M}_T}$ at ν intermediate points $\{x_j^{\ell}\}_{j=1}^{\nu} \subset \mathcal{M}_T$, parallel-transport each tangent vector back to $T_{\Theta_{1:T}^{\ell}}\mathcal{M}_T$, takes a linear combination with weights b_i , and applies the exponential map at $\Theta_{1:T}^{\ell}$. By the block-diagonal structure above, this procedure executes independently and simultaneously on each of the T factors $\mathbb{S}_{++}^p$. On factor i , the iterates are exactly those produced by the same scheme applied to the single-matrix Riemannian ODE $\dot{\Theta}_i = u_{\theta, i}^{\mathbb{S}_{++}^p}$ with initial condition Θ_i^0 , where $u_{\theta, i}^{\mathbb{S}_{++}^p}$ is the i -th block of $u_{\theta}^{\mathcal{M}_T}$. Proposition 3 of Collas et al. [45] applied to this i -th factor yields

$$\Theta_i^{\ell} = \phi^{-1}(z_i^{\ell}) \quad \text{for all } \ell \in \mathbb{N},$$

where $\{z_i^{\ell}\}$ is the corresponding Euclidean RK trajectory on the i -th block. Stacking across $i = 1, \dots, T$ gives (13).

Slice-wise SPD. Slice-wise SPD validity follows from $\Theta_i^{\ell} = \phi^{-1}(z_i^{\ell}) \in \mathbb{S}_{++}^p$ for every ℓ , as in the proof of Proposition 4. $\square$

E.6 Proof of Proposition 2 (standardised diffeomorphism)

In Section 3.1, each log-Euclidean coordinate is standardised using training statistics $\mu \in \mathbb{R}^d$ and $\sigma \in \mathbb{R}_{>0}^d$ broadcast across the trajectory index. Define

$$\Psi : E \rightarrow E, \quad \Psi(z)_{i,k} := \frac{z_{i,k} - \mu_k}{\sigma_k}, \quad \Psi^{-1}(\tilde{z})_{i,k} = \mu_k + \sigma_k \tilde{z}_{i,k}. \quad (44)$$

Proof of Proposition 2. Diffeomorphism. Ψ is an affine bijection of E with constant Jacobian $D\Psi(z) = \text{diag}(\sigma_1^{-1}, \dots, \sigma_d^{-1})$ broadcast across the trajectory index i ; since $\sigma_k > 0$, both Ψ and Ψ^{-1} are smooth. A composition of two global diffeomorphisms is a global diffeomorphism, so $\tilde{\Phi} = \Psi \circ \Phi$ is one.

Pullback metric. By the chain rule, $D\tilde{\Phi}(\Theta_{1:T}) = D\Psi(\Phi(\Theta_{1:T})) \circ D\Phi(\Theta_{1:T})$. For $\xi, \eta \in T_{\Theta_{1:T}}\mathcal{M}_T$,

$$\begin{aligned} (\tilde{\Phi}^* g_E)_{\Theta_{1:T}}(\xi, \eta) &= g_E(D\Psi(D\Phi[\xi]), D\Psi(D\Phi[\eta])) \\ &= \sum_{i,k} \sigma_k^{-1} [D\Phi[\xi]]_{i,k} \sigma_k^{-1} [D\Phi[\eta]]_{i,k} \\ &= \sum_{i=1}^T \sum_{k=1}^d \sigma_k^{-2} [D\phi(\Theta_i)[\xi_i]]_k [D\phi(\Theta_i)[\eta_i]]_k, \end{aligned}$$

where the last equality uses the block-diagonal form (38). This is (7).

Propositions 3–5 were proved using only two facts: Φ is a global diffeomorphism, and the pullback metric is $\Phi^* g_E$. Both of these hold for $\tilde{\Phi}$, with the pullback metric now (7). The identities (37) extend to $\tilde{\Phi}$ since they are general chain-rule facts about any global diffeomorphism. Hence the proofs go through with $\Phi \rightarrow \tilde{\Phi}$. $\square$

Remark 2 (Why standardisation matters in practice). Although $\tilde{\Phi}$ is geometrically equivalent to Φ under the rescaled metric (7), the choice of embedding matters in practice because the CFM source distribution $\mathcal{N}(0, I)$ is defined with respect to the unrescaled Euclidean inner product on E . The standardisation (44) aligns the source covariance with the per-coordinate variance of the target, removing the under-dispersion observed in the design-choice summary of Table 4.

E.7 Proposition 6 (CFM equivalence under arbitrary couplings) and its proof

The forecasting variants of Section 3.4 replace the independent Gaussian coupling, one by recentring the target and one by making the source depend on an extrapolation of the observed history. The following result shows the Riemannian/Euclidean equivalence survives any such coupling, so neither construction weakens the guarantees of Appendix E.

Proposition 6 (CFM equivalence under arbitrary couplings). *Fix a history $z_{1:K} \in E^K$ and a label y . Let $\pi(z_0^f, z_1^f \mid z_{1:K}, y)$ be any joint probability measure on $E_f \times E_f$ with finite second moment, and let $u_\theta^{E_f}(s, z^f, z_{1:K}, y)$ be the Euclidean field corresponding, via the pullback identity*

$$u_\theta^{E_f}(s, z^f, z_{1:K}, y) := D\tilde{\Phi}_f(\tilde{\Phi}_f^{-1}(z^f)) \left[u_\theta^{\mathcal{M}_{T-K}}(s, \tilde{\Phi}_f^{-1}(z^f), z_{1:K}, y) \right], \quad (45)$$

to a Riemannian field $u_\theta^{\mathcal{M}_{T-K}}$ on $(\mathcal{M}_{T-K}, \tilde{\Phi}_f^ g_{E_f})$. Then the Riemannian CFM loss along the component-wise log-Euclidean geodesic γ^f between $\tilde{\Phi}_f^{-1}(z_0^f)$ and $\tilde{\Phi}_f^{-1}(z_1^f)$ equals*

$$\mathcal{L}_{\text{forecast}}^{\mathcal{M}_{T-K}}(\theta) = \mathbb{E}_{s, y, z_{1:K}, (z_0^f, z_1^f) \sim \pi} \left\| u_\theta^{E_f}(s, (1-s)z_0^f + sz_1^f, z_{1:K}, y) - (z_1^f - z_0^f) \right\|_{E_f}^2. \quad (46)$$

This holds in particular for the independent Gaussian coupling, for the mean-centred residual coupling of (20)–(21), and for the random-walk random-walk coupling $\pi = p_{\text{RW}}(z_0^f \mid z_{1:K}) p_{\text{data}}(z_1^f \mid z_{1:K}, y)$ of (23).

Proof of Proposition 6. The argument is the trajectory analogue of Proposition 3, restricted to the future block and conditioned on the history, and the point of it is that nothing anywhere requires $z_0^f \perp z_1^f$.

Pointwise integrand identity. Fix any realisation $(s, y, z_{1:K}, z_0^f, z_1^f)$ with $z_0^f, z_1^f \in E_f$, and set

$$z_s^f := (1-s)z_0^f + sz_1^f, \quad \gamma^f(s) := \tilde{\Phi}_f^{-1}(z_s^f). \quad (47)$$

Since $\tilde{\Phi}_f$ is a global diffeomorphism with pullback metric on $\mathcal{M}_{T-K}$ (Proposition 2 applied to the future block), γ^f is the geodesic on $(\mathcal{M}_{T-K}, \tilde{\Phi}_f^* g_{E_f})$ between $\tilde{\Phi}_f^{-1}(z_0^f)$ and $\tilde{\Phi}_f^{-1}(z_1^f)$, with velocity $\dot{\gamma}^f(s) = D\tilde{\Phi}_f^{-1}(z_s^f)[z_1^f - z_0^f]$. By exactly the calculation used for Proposition 3, applied to $\tilde{\Phi}_f$ and the field (45),

$$\left\| u_\theta^{\mathcal{M}_{T-K}}(s, \gamma^f(s), z_{1:K}, y) - \dot{\gamma}^f(s) \right\|_{\gamma^f(s)}^2 = \left\| u_\theta^{E_f}(s, z_s^f, z_{1:K}, y) - (z_1^f - z_0^f) \right\|_{E_f}^2. \quad (48)$$

Equation (48) holds for every fixed pair (z_0^f, z_1^f) , with no probabilistic structure imposed on that pair. The derivation uses only the chain rule, the isometry of $D\tilde{\Phi}_f$ between tangent spaces, and the algebraic identity $D\tilde{\Phi}_f \circ D\tilde{\Phi}_f^{-1} = \text{Id}_{E_f}$. The conditioning variable $z_{1:K}$ enters only as an argument of the velocity field on both sides and does not touch the pullback geometry.

Taking expectations of both sides of (48) under $s \sim \mathcal{U}[0, 1]$, $y \sim \pi_Y$, $z_{1:K} \sim \pi_{\text{hist}}$ and $(z_0^f, z_1^f) \sim \pi(\cdot, \cdot \mid z_{1:K}, y)$, the left-hand side becomes the Riemannian CFM loss on $\mathcal{M}_{T-K}$ with source and target marginals $(\tilde{\Phi}_f^{-1})_{\#}\pi_0$ and $(\tilde{\Phi}_f^{-1})_{\#}\pi_1$, while the right-hand side is (46). $\square$

E.8 Proof of Proposition 7 (boundary term as squared log-Euclidean distance)

The losses of Eqs. (18) and (14) are written in the standardised embedding. We identify their counterparts in the log-Euclidean geometry on $\mathbb{S}_{++}^p$ (Proposition 7 for the boundary term, Proposition 8 for the temporal regulariser).

Proposition 7 (Boundary term as squared log-Euclidean distance). *Following the subscript convention of Section 3 (first index: flow time; second index: window index), let $\Theta_{K+1} := \tilde{\Phi}^{-1}(z_{1,1}^f) \in \mathbb{S}_{++}^p$ and $\hat{\Theta}_{K+1} := \tilde{\Phi}^{-1}(\hat{z}_{1,1}^f) \in \mathbb{S}_{++}^p$ be the decoded true and predicted first-future precision matrices (window index $i = K+1$, flow time $s = 1$). Then*

$$\left\| \hat{z}_{1,1}^f - z_{1,1}^f \right\|_2^2 = \sum_{k=1}^d \sigma_k^{-2} (\phi(\hat{\Theta}_{K+1})_k - \phi(\Theta_{K+1})_k)^2, \quad (49)$$

which reduces, when $\sigma_k \equiv 1$, to the squared log-Euclidean distance $d_{\text{LE}}^2(\hat{\Theta}_{K+1}, \Theta_{K+1})$.

Proof of Proposition 7. The map Ψ^{-1} is affine with diagonal action: $\Psi^{-1}(\tilde{z})_{i,k} - \Psi^{-1}(\tilde{z}')_{i,k} = \sigma_k(\tilde{z}_{i,k} - \tilde{z}'_{i,k})$ component-wise. Applied to $\tilde{z} = \hat{z}_{1,1}^f$ and $\tilde{z}' = z_{1,1}^f$ (both elements of $\mathbb{R}^d$, indexed by k),

$$(\Psi^{-1}(\hat{z}_{1,1}^f) - \Psi^{-1}(z_{1,1}^f))_k = \sigma_k(\hat{z}_{1,1}^f - z_{1,1}^f)_k. \quad (50)$$

The unstandardised differences are precisely the log-Euclidean residuals: $\Psi^{-1}(\hat{z}_{1,1}^f) = \phi(\hat{\Theta}_{K+1})$ and $\Psi^{-1}(z_{1,1}^f) = \phi(\Theta_{K+1})$, by definition of $\tilde{\Phi}^{-1}$ and the fact that $\tilde{\Phi}^{-1} = \phi^{-1} \circ \Psi^{-1}$ applied slice-by-slice. Therefore

$$\sigma_k(\hat{z}_{1,1}^f - z_{1,1}^f)_k = \phi(\hat{\Theta}_{K+1})_k - \phi(\Theta_{K+1})_k,$$

and squaring and dividing by σ_k^2 gives the k -th term of (49); summing over k yields the identity. When $\sigma_k \equiv 1$, the right-hand side equals $\|\phi(\hat{\Theta}_{K+1}) - \phi(\Theta_{K+1})\|_2^2 = \|\log \hat{\Theta}_{K+1} - \log \Theta_{K+1}\|_F^2 = d_{\text{LE}}^2(\hat{\Theta}_{K+1}, \Theta_{K+1})$ by (33). $\square$

E.9 Proof of Proposition 8 (temporal regulariser as Riemannian smoothness)

The temporal regulariser (14) is stated in the standardised embedding; Proposition 8 identifies its geometric meaning on $\mathbb{S}_{++}^p$.

Proposition 8 (Temporal regulariser as Riemannian smoothness). *Let $\hat{\Theta}_i := \tilde{\Phi}^{-1}(\hat{z}_{1,i}) \in \mathbb{S}_{++}^p$ for $i = 1, \dots, T$ be the decoded predicted-endpoint trajectory.*

(i) *With $\psi_E(\cdot) = \|\cdot\|_2^2$ (used in the smooth and group-lasso TVGL settings),*

$$\frac{1}{T-1} \sum_{i=2}^T \|\hat{z}_{1,i} - \hat{z}_{1,i-1}\|_2^2 = \frac{1}{T-1} \sum_{i=2}^T \sum_{k=1}^d \sigma_k^{-2} (\phi(\hat{\Theta}_i)_k - \phi(\hat{\Theta}_{i-1})_k)^2, \quad (51)$$

which is, up to per-coordinate scaling, the average squared log-Euclidean distance between consecutive predicted matrices.

(ii) With $\psi_E(\cdot) = \|\cdot\|_1$ (used in the ℓ_1 TVGL setting),

$$\sum_{i=2}^T \|\hat{z}_{1,i} - \hat{z}_{1,i-1}\|_1 = \sum_{i=2}^T \sum_{k=1}^d \sigma_k^{-1} |\phi(\hat{\Theta}_i)_k - \phi(\hat{\Theta}_{i-1})_k|, \quad (52)$$

i.e. a weighted ℓ_1 norm of vec_{lt} -vectorised log-Euclidean increments $\log \hat{\Theta}_i - \log \hat{\Theta}_{i-1}$, not of raw precision increments $\hat{\Theta}_i - \hat{\Theta}_{i-1}$.

Proof of Proposition 8. (i) By the diagonal action of Ψ^{-1} (Eq. (50)), applied slice-by-slice with $(\hat{z}_{1,i-1}, \hat{z}_{1,i})$ in place of $(z_{1,1}^f, \hat{z}_{1,1}^f)$,

$$\|\hat{z}_{1,i} - \hat{z}_{1,i-1}\|_2^2 = \sum_k \sigma_k^{-2} (\phi(\hat{\Theta}_i)_k - \phi(\hat{\Theta}_{i-1})_k)^2.$$

Summing over $i = 2, \dots, T$ and dividing by $T - 1$ gives (51).

(ii) By the same diagonal action (50), $\sigma_k(\hat{z}_{1,i} - \hat{z}_{1,i-1})_k = \phi(\hat{\Theta}_i)_k - \phi(\hat{\Theta}_{i-1})_k$ componentwise, so taking absolute values and dividing by σ_k , $|\hat{z}_{1,i,k} - \hat{z}_{1,i-1,k}| = \sigma_k^{-1} |\phi(\hat{\Theta}_i)_k - \phi(\hat{\Theta}_{i-1})_k|$, and summing over k and i gives (52). $\square$

E.10 Summary

Propositions 1–7 together establish that the entire TVGL–CFM pipeline admits the same pullback equivalence as the single-matrix case of Collas et al. [45]:

- Trajectory-level training, ODE integration, and explicit Runge–Kutta sampling reduce to their Euclidean counterparts on $\mathbb{R}^{T \times d}$ (Props 3, 4, and 5).
- Per-coordinate standardisation preserves the equivalence under a rescaled product log-Euclidean metric (Prop 2).
- History-conditioned forecasting with *any* joint source–target coupling, including both couplings of Section 3.4, preserves the equivalence (Prop 6).
- The boundary term (18) and the ℓ_2 temporal regulariser are, respectively, a squared log-Euclidean distance and the average squared log-Euclidean increment between consecutive predicted matrices; the ℓ_1 temporal regulariser is a log-Euclidean equivalent for TVGL’s ℓ_1 evolutionary penalty (Props 8 and 7).

Every decoded trajectory $\Theta_{1:T} = \tilde{\Phi}^{-1}(\tilde{z})$ therefore lies slice-wise in $\mathbb{S}_{++}^p$ by construction, without projection.

F Design choices

Table 4 summarises the main design choices and their effect. The embedding standardisation is validated quantitatively by the generation and forecasting results above; the remaining choices are motivated in Section 3 and Appendix K.

Table 4: Design choices and their effect.

Choice	Effect
Standardise embedding (§3.2)	Prevents under-dispersion; lowers discriminator AUC and corrects spectrum std-ratio.
OAS shrinkage covariance (§3.1)	Well-conditioned targets when $n \sim p$; stabilises TVGL and downstream metrics.
Scale-aware λ, β (§2.3)	Preserves class signal and genuine dynamics in the targets (avoids over-smoothing/over-sparsifying).
Covariance-space CAS (§K)	Measures the discriminative second-order structure; diagnoses where signal is lost.
Data-dependent couplings (§3.4)	The mean-centred residual coupling recentres the target on a training-set future mean and is stronger on EEG. The random-walk coupling instead replaces the source by an extrapolation of recent graph velocity.
Boundary term, forecasting (§3.3)	Smooth, correct history to future handoff; avoids collapse to persistence and discontinuities.
Block (non-autoregressive) future (§3.3)	Future windows attend jointly; avoids autoregressive error accumulation over the horizon.

G Sensitivity to TVGL Regularization Parameters

TVGL introduces two regularization parameters that can significantly influence the target precision trajectories used by TVGL-CFM: the sparsity parameter λ and the temporal regularization parameter β . Because the proposed generative model is trained on TVGL-estimated trajectories, it is important to verify that its performance is not tied to a narrow choice of these parameters. We therefore performed a controlled sensitivity study on BNCI2014_001 using the same pooled cross-session generation protocol as in the main experiments. We varied

$$\lambda \in \{0.05, 0.10, 0.20\}, \quad \beta \in \{0.10, 0.20, 0.40\},$$

which spans one-half to twice the values used in the main experiments, $(\lambda, \beta) = (0.10, 0.20)$. All other preprocessing, TVGL, model, training, and evaluation settings were held fixed.

Table 5 reports both the discriminative quality of the corresponding real TVGL targets (*TVGL oracle*) and the quality of trajectories generated by TVGL-CFM. Across the full 3×3 grid, the oracle itself is fairly stable: its AUC varies only from 0.784 to 0.801 and its F1 score from 0.704 to 0.726. This indicates that the discriminative structure of the inferred TVGL trajectories is preserved across a substantial neighborhood of the default estimator configuration.

TVGL-CFM is similarly stable. Across all nine settings, generated trajectories attain CAS AUC values between 0.684 and 0.736, F1 scores between 0.616 and 0.679, and relative Fréchet distances between 1.041 and 1.133. Every setting yields an SPD fraction of 1.00. The default configuration $(\lambda, \beta) = (0.10, 0.20)$ attains the highest CAS AUC in the grid, 0.736, while the neighboring setting $(0.10, 0.10)$ yields the highest F1 score, 0.679. The lowest relative Fréchet distance, 1.041, occurs for $(0.20, 0.10)$, although this setting also exhibits the weakest CAS performance, suggesting a trade-off between global distributional similarity and preservation of class-discriminative structure.

Increasing temporal regularization to $\beta = 0.40$ consistently raises the relative Fréchet distance compared with the corresponding lower β settings, while stronger sparsification ($\lambda = 0.20$) can reduce discriminative performance when combined with weak temporal regularization. Nevertheless, the absence of a sharp performance collapse across the grid indicates that TVGL-CFM does not rely on a narrowly tuned TVGL configuration. These results support the use of the moderate default $(\lambda, \beta) = (0.10, 0.20)$ employed in the main experiments.

Table 5: Sensitivity of TVGL–CFM generation to the TVGL sparsity parameter λ and temporal regularization parameter β on BNCI2014_001 under the pooled cross-session split. Oracle AUC/F1 are computed from the corresponding real TVGL trajectories. CFM AUC/F1 are CAS scores obtained from generated trajectories, and Rel-FD is the relative Fréchet distance to the real held-out distribution. All generated precision matrices are SPD. The default TVGL configuration used in the main experiments is shown in bold.

λ	β	Oracle AUC	Oracle F1	CFM AUC	CFM F1	Rel-FD	SPD
0.05	0.10	0.785	0.708	0.719	0.658	1.055	1.00
0.05	0.20	0.787	0.712	0.726	0.656	1.073	1.00
0.05	0.40	0.801	0.714	0.711	0.648	1.119	1.00
0.10	0.10	0.784	0.704	0.735	0.679	1.056	1.00
0.10	0.20	0.790	0.710	0.736	0.668	1.066	1.00
0.10	0.40	0.795	0.726	0.716	0.668	1.124	1.00
0.20	0.10	0.797	0.714	0.684	0.616	1.041	1.00
0.20	0.20	0.795	0.719	0.722	0.656	1.093	1.00
0.20	0.40	0.796	0.717	0.715	0.653	1.133	1.00

H Robustness to the Temporal Graph Estimator

TVGL–CFM uses precision-matrix trajectories estimated by TVGL as its learning targets. Although TVGL provides a statistically principled and temporally coupled representation of dynamic networks, this raises the question of whether the downstream conditional flow-matching model depends specifically on the TVGL estimator. To test this directly, we replace TVGL with LOGGLE [9], a distinct estimator of time-varying Gaussian graphical models, while leaving the remainder of the trajectory-generation pipeline unchanged.

LOGGLE [9] estimates a sequence of locally time-varying precision matrices by combining kernel-localised covariance estimation with group sparsity across a temporal neighbourhood. For a target time point t_k , let

$$\mathcal{N}_{k,d} = \{i : |t_i - t_k| \leq d\},$$

with $m_k = |\mathcal{N}_{k,d}|$. Given locally smoothed covariance estimates $\hat{\Sigma}(t_i)$, the likelihood formulation of LOGGLE estimates the precision matrices within this neighbourhood by solving

$$\min_{\{\Omega(t_i) \succ 0\}_{i \in \mathcal{N}_{k,d}}} \frac{1}{\sqrt{m_k}} \sum_{i \in \mathcal{N}_{k,d}} \left[\text{tr} \left(\Omega(t_i) \hat{\Sigma}(t_i) \right) - \log \det \Omega(t_i) \right] + \lambda_L \sum_{u \neq v} \sqrt{\sum_{i \in \mathcal{N}_{k,d}} \Omega_{uv}(t_i)^2}.$$

Unlike TVGL, which imposes an evolutionary penalty directly between adjacent precision matrices, LOGGLE constructs kernel-localised covariance estimates and encourages a common sparse edge structure within a local temporal neighbourhood. It therefore provides a substantially different mechanism for estimating dynamic precision trajectories.

We use the Epanechnikov kernel

$$K_h(x) = \frac{3}{4} \left(1 - \left(\frac{x}{h} \right)^2 \right) \mathbb{I}(|x| \leq h),$$

with normalised kernel weights for local covariance estimation. We implement the likelihood LOGGLE objective of Yang and Peng [9], using their temporal-neighbourhood, Epanechnikov-kernel, and group-sparsity definitions. Optimisation is performed with a batched GPU ADMM solver; this changes only the numerical execution and not the underlying LOGGLE objective.

Experimental setup. We evaluate LOGGLE–CFM on BNCI2014_001 under the same pooled cross-session generation protocol used in the main experiments. The training and held-out test splits each contain 1296 EEG trials, with $p = 22$ channels and $N = 500$ temporal samples per trial. Each trial is represented by a trajectory of $T = 20$ precision matrices, with graph locations placed at the centres of the same 20 temporal bins used in the main pipeline.

For LOGGLE, we use the reference bandwidth rule

$$h = 0.8N^{-1/5},$$

which gives $h = 0.230832$ for $N = 500$, together with temporal neighbourhood width $d = 0.2$ and sparsity parameter $\lambda_L = 0.25$. We fit on the correlation scale, globally centre each channel (detrend=False), and omit the optional post-estimation refitting stage (refit=False). The result-

Table 6: Robustness of conditional flow matching to replacement of TVGL with LOGGLE on BNCI2014_001 under the pooled cross-session split. The LOGGLE oracle reports classification using real LOGGLE trajectories, whereas LOGGLE-CFM reports CAS obtained by training on generated LOGGLE trajectories and evaluating on the held-out real LOGGLE test trajectories. Rel-FD is the Fréchet distance normalised by the corresponding real train–test distance.

Method	CAS AUC $\uparrow$	CAS F1 $\uparrow$	Rel-FD $\downarrow$	SPD $\uparrow$
Real LOGGLE oracle	0.803	0.737	–	1.00
LOGGLE-CFM	0.770	0.680	1.686	1.00

ing LOGGLE precision matrices replace the TVGL targets directly; no additional inverse-covariance estimator is applied.

All downstream components are held fixed. Each LOGGLE precision matrix is mapped to the same log-Euclidean SPD representation used by TVGL-CFM, complete 20-matrix trajectories are standardised using training-set statistics, and the same class-conditional trajectory flow model is trained in this representation for 1000 epochs. Generation is evaluated using the same train-on-generated/test-on-real classifiability score (CAS), relative Fréchet distance, SPD validity, spectral diagnostics, and temporal-dynamics statistics used in the main generation experiments. Thus, the only substantive change in the representation pipeline is the temporal graph estimator that produces the real target trajectories.

Results. Table 6 shows that replacing TVGL with LOGGLE preserves strong class-discriminative information in the generated trajectories. The real LOGGLE trajectories attain an oracle AUC/F1 of 0.803/0.737, while LOGGLE-CFM achieves a train-on-generated/test-on-real CAS AUC of 0.770 and F1 of 0.680. The AUC is therefore only 0.033 below the corresponding real-trajectory oracle, despite changing the temporal graph estimator and learning the complete trajectory distribution generatively.

The structural and spectral diagnostics are also favourable. All generated precision matrices remain symmetric positive definite, yielding an SPD fraction of 1.00. The pooled spectral Wasserstein distance between generated and real training trajectories is 0.0117, while the generated-to-real spectral standard-deviation ratio is 1.002, indicating close agreement in the distribution and spread of eigenvalues. Temporal variation is similarly preserved, with a temporal-dynamics ratio of 1.47 for generated trajectories compared with 1.57 for the real LOGGLE trajectories.

The absolute Fréchet distance between generated and held-out real LOGGLE trajectories is 1229.82, compared with a real train–test reference distance of 729.42, resulting in a relative Fréchet distance of 1.686. This indicates that full high-dimensional distribution matching remains more difficult under the LOGGLE target representation, while class-discriminative utility, SPD validity, spectral structure, and temporal variation are nevertheless strongly preserved.

Overall, this experiment provides an estimator-level robustness check complementary to the TVGL (λ, β) sensitivity analysis in Appendix G. The sensitivity study tests robustness within the TVGL estimator family, whereas the LOGGLE experiment changes the temporal graph-learning mechanism itself. The resulting CAS AUC/F1 of 0.770/0.680, full SPD validity, closely matched spectral statistics, and preserved temporal dynamics demonstrate that the conditional flow-matching formulation is not tied specifically to TVGL-estimated trajectories and can operate successfully on precision trajectories produced by a distinct time-varying graphical-model estimator.

I Complexity Analysis

A limitation of the log-Euclidean representation is that its coordinate dimension grows quadratically with the number of graph nodes. For a $p \times p$ SPD precision matrix, the vectorized lower-triangular log-Euclidean representation has dimension

$$d(p) = \frac{p(p+1)}{2}, \quad (53)$$

so a trajectory of T precision matrices contains $T d(p)$ Euclidean coordinates. In addition to this representation growth, the dense matrix eigendecompositions required by TVGL and by the matrix logarithm become increasingly expensive as p increases.

To quantify the practical scaling regime of the complete pipeline, we performed a controlled synthetic benchmark using multivariate Gaussian time series with increasing dimensionality. For each graph size $p \in \{16, 22, 32, 64, 96, 128, 192, 256, 384, 512\}$, we generated 128 training and 64 test

Table 7: End-to-end computational scaling of TVGL–CFM on synthetic multivariate Gaussian time series. $d = p(p + 1)/2$ is the log-Euclidean dimension of one precision matrix and Td is the dimension of the complete $T = 20$ trajectory. CFM time/epoch is averaged over 15 training epochs. Peak VRAM refers to peak allocated GPU memory during CFM training. All configurations completed successfully.

p	d	Td	OAS (s)	TVGL (s)	Log map (s)	CFM s/epoch	Peak VRAM (GiB)
16	136	2,720	0.323	1.864	0.146	0.178	0.135
22	253	5,060	0.013	1.832	0.275	0.160	0.138
32	528	10,560	0.014	1.807	0.670	0.100	0.143
64	2,080	41,600	0.022	3.337	7.263	0.310	0.175
96	4,656	93,120	0.033	5.275	13.365	0.323	0.220
128	8,256	165,120	0.161	6.991	29.459	0.362	0.291
192	18,528	370,560	0.125	27.192	78.456	0.361	0.494
256	32,896	657,920	0.144	41.431	176.238	0.535	0.785
384	73,920	1,478,400	0.167	104.774	416.382	0.713	1.606
512	131,328	2,626,560	0.209	233.199	991.754	0.790	2.756

trajectories, each consisting of $T = 20$ windows with 128 multivariate observations per window. The observations were passed through the same processing chain used in the main experiments: windowed OAS covariance estimation, TVGL inference, log-Euclidean embedding, and TVGL–CFM training and sampling. We fixed the TVGL parameters to $\lambda = 0.1$, $\beta = 0.2$, the group temporal penalty, and 150 ADMM iterations. For the scaling benchmark, TVGL–CFM was trained for 15 epochs; this experiment is intended to measure computational cost rather than synthetic-data generation quality. Table 7 reports the resulting dimensionality, runtime, and GPU-memory requirements. All tested graph sizes completed successfully through $p = 512$. At the largest setting, each precision matrix is represented by 131,328 log-Euclidean coordinates and the complete 20-window trajectory contains 2,626,560 coordinates. Despite this representation size, TVGL–CFM training itself remained moderate, requiring approximately 0.79 s per epoch and 2.76 GiB of peak allocated GPU memory. The dominant cost at larger graph sizes instead arises from the dense SPD preprocessing stages. At $p = 512$, TVGL inference required 233.20 s and the log-Euclidean map required 991.75 s, compared with 11.84 s for all 15 TVGL–CFM training epochs. The log-map stage therefore becomes the main computational bottleneck as graph size increases. Figure 4 illustrates this separation clearly: OAS covariance estimation and CFM sampling remain small across the tested range, whereas TVGL and especially the matrix logarithm increase rapidly with p .

Although the quadratic representation size makes very large graphs increasingly expensive, the present implementation is not restricted to the 13–22 node EEG networks considered in the main experiments: the complete benchmark remained feasible through 512 nodes. The main practical limitation in this regime is the cost of dense TVGL and matrix-function operations rather than flow-matching training itself. Scaling to substantially larger graphs, particularly graphs with thousands of nodes, would likely require sparse, low-rank, block-structured, or approximate matrix-function representations.

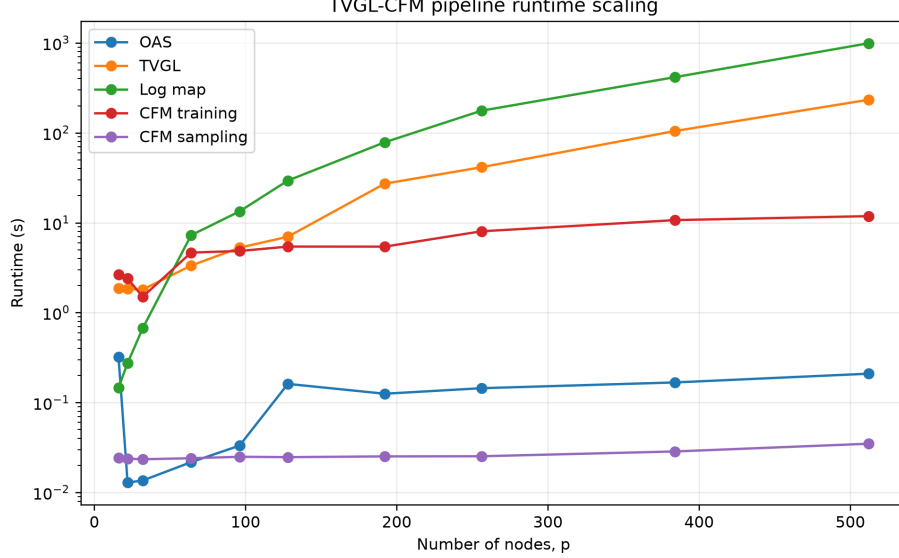


Figure 4: Runtime scaling of the complete TVGL–CFM pipeline with increasing graph size. The vertical axis is logarithmic. The log-Euclidean matrix logarithm becomes the dominant cost at larger p , followed by TVGL inference, while OAS covariance estimation, CFM training, and CFM sampling remain substantially smaller over the tested range.

J Experimental Details

This appendix gives the full experimental configuration for the generation results (Table 1), the EEG forecasting results (Table 2), and the cross-domain forecasting results (Table 3 and Appendix N). All settings are those used to produce the reported numbers; hyperparameters were fixed across datasets within each experiment unless noted. Code will be released on acceptance.

J.1 Datasets and preprocessing

EEG motor imagery. We use four publicly available motor-imagery datasets from the MOABB benchmark [70]: BNCI2014_001 ($p = 22$ channels), BNCI2014_002 ($p = 15$), BNCI2015_001 ($p = 13$), and Zhou2016 ($p = 14$), each reduced to a two-class (right-hand vs. feet/left-hand) paradigm. Raw recordings are band-pass filtered to 4–38 Hz and resampled to 128 Hz, EEG channels are converted to μV , and each trial is split into $T = 20$ contiguous windows. Per-channel z -scoring is applied over time before windowing. We report a *pooled cross-session* split (a held-out session/run per subject) for generation and forecasting; means and standard deviations are computed over held-out trials.

Cross-domain dynamical systems. Three simulated multivariate systems, exported as coarse-grained CSV trajectories, probe domain generality: **Lorenz**, a network of diffusively coupled Lorenz oscillators (deterministic chaos); **MacArthur**, a MacArthur consumer–resource competition system (ecological population dynamics); and **Hopfield**, a continuous Hopfield associative-memory network relaxing toward stored attractors. Each system is a single long multivariate time series with no class label. We form overlapping pseudo-trials by sliding a window of $\text{WIN_SIZE} = 15$ samples, using $T = K + h$ windows per trial with a fixed history of $K = 15$ windows and forecast horizons $h \in \{3, 5, 7\}$; the $h = 5$ setting is the one reported in the main table (Table 3) and the full sweep over $h \in \{3, 5, 7\}$ is given in Appendix N. A temporal train/test split (train fraction = 0.7) with a gap = 2 windows between the training and test segments prevents leakage across the boundary; node signals are normalised per coordinate before covariance estimation.

K Evaluation

We use a *trajectory-respecting* protocol: each sample is the whole chain (for forecasting, the future block), embedded per window and concatenated into a single vector, so metrics are sensitive to temporal arrangement and not only to the marginal set of matrices.

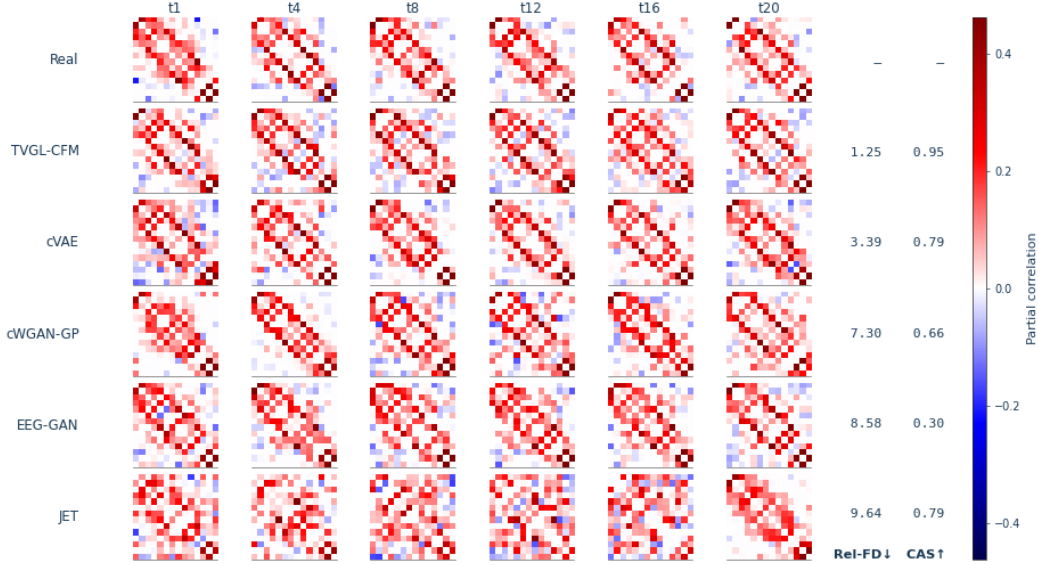


Figure 5: Qualitative comparison of TVGL precision-trajectory generation. Each row shows selected time windows from a generated or real TVGL trajectory, displayed as partial-correlation matrices. The TVGL-CFM row is visualized using a sparse post-processing readout for interpretability, while the reported Rel-FD and CAS values correspond to the unsparisified TVGL-CFM output used in quantitative evaluation. Lower Rel-FD indicates better distributional fidelity; higher CAS indicates better class-conditional utility.

Distributional quality (both tasks). We report a relative Fréchet distance (Rel-FD) between generated and real trajectories in the diffeomorphic embedding.

Classification-accuracy score, CAS (both tasks). Following the train-on-synthetic, test-on-real protocol [84], we train a compact multi-layer perceptron (MLP) probe on generated trajectories and evaluate it on real test data. Rather than using logistic regression, we employ a small MLP to retain a lightweight and efficient evaluation while allowing mild nonlinear decision boundaries. We also report a TVGL-precision oracle, obtained by training the same probe on real training trajectories and evaluating it on held-out real trajectories; the oracle-to-model gap therefore measures the discriminative information lost during generation. By (2), CAS features may equivalently be represented in covariance space.

Forecasting-specific. We add (i) direct error: standardised-embedding RMSE and Frobenius RMSE on the future precision matrices against the true future. A natural baseline is *persistence* (copy the last observed window across the horizon), and for the mean-centred residual coupling we also report the *training mean alone*, i.e. the zero-residual baseline. A useful forecaster must beat both on direct error while also matching the future distribution and dynamics.

L Implementation details and baselines

L.1 Embedding and TVGL-CFM architecture

Each precision trajectory is embedded window-wise through the log-Euclidean chart (Eq. (5)) and standardised per coordinate using training-set statistics (Eq. (6)). The velocity field u_θ (Eq. (9)) is a 4-layer Transformer *encoder* with model width $d_{\text{model}} = 256$, 8 attention heads, feed-forward width $4d_{\text{model}}$, GELU activations and no dropout. Flow time s , window time τ , and the class label enter through independent random-Fourier-feature embeddings ($N_{\text{FF}} = 12$ frequencies for EEG, 8 for the cross-domain systems) followed by an MLP, plus a learned class embedding. Training uses AdamW [75] (learning rate 5×10^{-4} , weight decay 10^{-4}), 500 epochs and batch size 256. The temporal-consistency weight is $\lambda_{\text{temp}} = 0.1$. Sampling integrates the flow ODE with an explicit RK4 solver (50 steps for EEG, 30 for the cross-domain systems). The forecasting models below use a *different* and smaller backbone; the two are not shared.

EEG forecasting architecture. The forecaster is a separate, smaller model: a 2-layer Transformer *encoder* for the history (Eq. (15)) and a 2-layer Transformer *decoder* for the future flow (Eq. (17)),

both with $d_{\text{model}} = 128$, 4 attention heads, feed-forward width $4 d_{\text{model}}$, GELU activations, norm-first blocks, no dropout, and $N_{\text{FF}} = 12$. The history encoder returns $H \in \mathbb{R}^{K \times d_{\text{model}}}$; all $K = 15$ ordered history tokens are retained. The future decoder conditions on H by cross-attention, so every future token can attend to every observed window, and future tokens also self-attend to one another with no causal target mask. Both couplings share this backbone and differ only in the CFM source/target construction.

Forecasting training and sampling. We train one forecaster per coupling. In the mean-centred residual variant (Eqs. (20)–(21)) the source is a unit-scale Gaussian in residual coordinates; the training mean, taken per class where labels exist, is estimated on the fitting portion of the training split only, a held-out portion is used for checkpoint selection. In the random-walk variant (Eqs. (22)–(23)) the source is built from $r = 3$ recent history increments with $\alpha_\mu = \alpha_\sigma = 1$. In both cases the CFM term is computed in the coupling’s own coordinates while the structural losses are evaluated on the reconstructed absolute future, with $\lambda_{\text{temp}} = 0.02$ and $\lambda_{\text{bnd}} = 0.10$. The final EEG forecasting benchmark uses batch size 256. Sampling uses 50 RK4 steps and an $M = 32$ -member ensemble; the point forecast is the ensemble mean in standardised log-Euclidean coordinates, decoded once through ϕ^{-1} , and the spread gives predictive uncertainty.

L.2 Baselines

We evaluate three families of baselines: *non-learned forecasting controls*, *direct TVGL-space models*, and *raw-signal generators/forecasters*.

Non-learned forecasting controls. These baselines require no learned dynamics model. TRAINING MEAN (NO FLOW) predicts m^f of Eq. (20) directly, i.e. the zero-residual baseline of Forecast TVGL–CFM, using the same class-conditioned mean estimated on the same training fitting subset. Because the mean is identical, the gap between this baseline and Forecast TVGL–CFM isolates the contribution of the learned history-conditioned residual transport. On the label-free simulated systems it is the global training-set future mean. PERSISTENCE copies the last observed precision matrix Θ_K across the full forecast horizon. LINEAR HISTORY-DRIFT extrapolates linearly in the standardised log-Euclidean chart from the recent history.

Direct TVGL-space forecasting baselines. To isolate the effect of conditional flow matching from the benefit of modelling precision trajectories directly, we compare against models trained on the same standardised log-Euclidean TVGL representation as TVGL–CFM. Each forecaster receives the same causal history $\tilde{z}_{1:K} \in \mathbb{R}^{K \times d}$ and predicts the same future block $\tilde{z}_{K+1:T} \in \mathbb{R}^{h \times d}$, with $d = p(p+1)/2$. Training statistics are computed from the training split only, and all predictions are decoded through the same inverse standardisation and log-Euclidean map.

Their outputs are predictions of $\tilde{z}_{K+1:T}$ itself rather than residuals relative to m^f . Stochastic baselines use $M = 32$ samples and form the point forecast by averaging the generated absolute future trajectories in standardised log-Euclidean coordinates before SPD decoding. All neural baselines are trained for 500 epochs.

- **TVGL–cVAE-Forecast:** a conditional VAE that generates the complete future TVGL block jointly from the observed TVGL history and class label. During training, the encoder receives the observed history together with the true absolute future and parameterises a Gaussian latent variable. The decoder maps the observed history, class label, and latent sample directly to $\hat{z}_{K+1:T} \in \mathbb{R}^{h \times d}$ in one pass. We use hidden width 512, latent dimension 64, KL weight 10^{-3} , and learning rate 10^{-3} .
- **TVGL–DDPM-Forecast:** a conditional diffusion model operating directly on the future TVGL block. Gaussian noise is added to the true future trajectory during training, and a Transformer denoiser conditioned on the complete observed history, class label, diffusion step, and future-window position learns the standard noise-prediction objective. At inference, reverse diffusion starts from Gaussian noise and directly generates a future trajectory in the standardised log-Euclidean chart. We use 50 diffusion steps, a linear variance schedule from 10^{-4} to 2×10^{-2} , and learning rate 5×10^{-4} .
- **DyGraphformer-T:** a graph-native deterministic forecaster adapted to the TVGL representation. Each observed log-Euclidean TVGL vector is reconstructed as a symmetric matrix whose rows provide node-wise connectivity features. A recurrent node state and multi-head spatial attention model dynamic interactions across windows, after which temporal Transformer layers aggregate the observed trajectory. The prediction head directly outputs the complete *absolute* $h \times d$ future

TVGL block. We use node width 128, 4 spatial heads, 4 temporal heads, and learning rate 5×10^{-4} .

Direct TVGL-space generation baselines. For generation, we use three additional baselines operating directly on complete standardised log-Euclidean TVGL trajectories:

- **TVGL-cVAE:** a class-conditional VAE that generates the complete absolute TVGL trajectory jointly from a Gaussian latent variable.
- **TVGL-DDPM:** a class-conditional diffusion model whose Transformer denoiser iteratively generates the complete absolute TVGL trajectory from noise.
- **Window-DIFFEO-CFM:** a conditional flow-matching model applied independently to each precision-matrix window. Its MLP velocity field is conditioned on the matrix representation, class, window time, and flow time, but there is no communication between windows. This tests whether the gain of TVGL-CFM comes from flow matching alone or from modelling the complete trajectory jointly.

Raw-signal generators and forecasters. Raw-signal baselines operate directly on multivariate EEG rather than on estimated precision trajectories. Their generated or forecast signals are subsequently passed through the *same* OAS+TVGL estimation pipeline used for the real data. This comparison therefore isolates direct TVGL-space modelling from the alternative strategy of first predicting or generating raw EEG and only then estimating its connectivity trajectory.

The predicted future EEG is concatenated with the real observed EEG history, the resulting completed trajectory is passed through the identical OAS+TVGL estimator, and only the future precision-matrix windows are retained for evaluation. For unconditional generation, the raw-signal models generate complete EEG trajectories, with class conditioning where applicable. The final neural comparisons use 500 training epochs unless otherwise determined by the corresponding source implementation.

Their configurations are:

- **Basic-CNN:** a deterministic convolutional encoder-decoder that receives the observed raw EEG history and directly predicts the complete future EEG segment. The forecasting model uses hidden width 256, learning rate 10^{-3} , and 500 epochs.
- **cVAE:** a conditional VAE operating on raw EEG, with latent dimension 64, hidden width 512, KL weight 10^{-3} , and learning rate 10^{-3} . For forecasting, the observed raw EEG history and class label condition generation of the complete future EEG segment. For unconditional generation, the class label provides the conditioning information.
- **cWGAN-GP:** a conditional Wasserstein GAN with gradient penalty operating on raw EEG. For forecasting, its generator receives the observed raw EEG history, class label, and latent noise and directly generates the future EEG segment. We use latent dimension 128, hidden width 256, $n_{\text{critic}} = 3$, gradient-penalty weight 10, and learning rate 2×10^{-4} .
- **EEG-GAN:** an EEG-specific progressive convolutional Wasserstein GAN based on the EEG-GAN design, using interpolation-based upsampling, temporal convolutions, PixelNorm, minibatch-standard-deviation features, a one-sided gradient penalty, and critic drift regularisation. We use latent dimension 200, $n_{\text{critic}} = 5$, gradient-penalty weight 10, and drift coefficient $\varepsilon = 10^{-3}$.
- **JET:** the official JET transformer flow-matching backbone for raw EEG [30]. We use the repository JET-B/16 configuration with patch length 200, transformer width 256, depth 6, 8 attention heads, exponential moving averages, and the source flow-matching objective. For forecasting, the observed EEG history is clamped throughout integration and JET generates the future EEG segment. The resulting raw EEG forecast is then concatenated with the observed history and passed through OAS+TVGL.

For all raw-signal baselines, generated signals are clipped to ± 5 training-set standard deviations before inverse standardisation, covariance estimation, and TVGL reconstruction.

M Seed variability for the EEG tables

Tables 1 and 2 report means over three seeds for legibility. The same numbers with their standard deviations are given below.

Table 8: Table 1 with standard deviations. Three-seed generation performance on EEG datasets, mean $\pm$ standard deviation across three random seeds. CAS AUC and F1 measure train-on-generated $\rightarrow$ test-on-real classifiability (higher is better), while Rel-FD is the Fréchet distance between generated and held-out real precision trajectories normalized by the real train–test reference (lower is better). Direct precision-space models generate TVGL trajectories directly; raw-EEG baselines generate signals that are subsequently mapped through the same OAS+TVGL pipeline.

Method	Zhou2016			BNCI2014_001			BNCI2014_002			BNCI2015_001			Mean		
	AUC	F1	Rel-FD	AUC	F1	Rel-FD	AUC	F1	Rel-FD	AUC	F1	Rel-FD	AUC	F1	Rel-FD
TVGL-oracle	0.965	0.900	–	0.794	0.722	–	0.777	0.717	–	0.762	0.692	–	–	–	–
TVGL–CFM	0.954$\pm$0.005	0.893$\pm$0.008	<u>1.101$\pm$0.092</u>	0.718$\pm$0.023	0.646$\pm$0.007	1.191$\pm$0.096	0.736$\pm$0.006	0.678$\pm$0.021	<u>1.022$\pm$0.020</u>	0.717$\pm$0.015	0.649$\pm$0.005	1.061$\pm$0.099	0.781$\pm$0.007	0.716$\pm$0.008	1.094$\pm$0.006
TVGL–cVAE	0.879 $\pm$ 0.016	0.799 $\pm$ 0.009	1.111 $\pm$ 0.002	<u>0.677$\pm$0.014</u>	<u>0.620$\pm$0.015</u>	1.938 $\pm$ 0.000	<u>0.680$\pm$0.018</u>	<u>0.621$\pm$0.016</u>	0.994$\pm$0.007	<u>0.693$\pm$0.009</u>	<u>0.635$\pm$0.009</u>	2.725 $\pm$ 0.008	<u>0.732$\pm$0.006</u>	<u>0.669$\pm$0.008</u>	1.692 $\pm$ 0.004
TVGL–DDPM	0.889 $\pm$ 0.017	0.792 $\pm$ 0.016	1.075$\pm$0.006	0.615 $\pm$ 0.004	0.571 $\pm$ 0.018	<u>1.572$\pm$0.008</u>	0.624 $\pm$ 0.066	0.587 $\pm$ 0.054	1.098 $\pm$ 0.015	0.603 $\pm$ 0.034	0.569 $\pm$ 0.019	<u>1.223$\pm$0.032</u>	0.683 $\pm$ 0.017	0.629 $\pm$ 0.019	<u>1.242$\pm$0.005</u>
Window-DIFFEO-CFM	<u>0.933$\pm$0.004</u>	<u>0.853$\pm$0.011</u>	1.208 $\pm$ 0.008	0.626 $\pm$ 0.010	0.596 $\pm$ 0.012	2.145 $\pm$ 0.007	0.661 $\pm$ 0.004	0.621 $\pm$ 0.017	1.386 $\pm$ 0.007	0.664 $\pm$ 0.007	0.593 $\pm$ 0.007	1.536 $\pm$ 0.013	0.721 $\pm$ 0.005	0.666 $\pm$ 0.007	1.569 $\pm$ 0.002
JET $\rightarrow$ TVGL	0.573 $\pm$ 0.067	0.398 $\pm$ 0.110	9.216 $\pm$ 0.326	0.508 $\pm$ 0.050	0.385 $\pm$ 0.081	11.374 $\pm$ 0.288	0.506 $\pm$ 0.015	0.356 $\pm$ 0.041	7.845 $\pm$ 0.191	0.505 $\pm$ 0.010	0.334 $\pm$ 0.001	31.738 $\pm$ 0.328	0.523 $\pm$ 0.026	0.368 $\pm$ 0.044	15.043 $\pm$ 0.260
cVAE $\rightarrow$ TVGL	0.880 $\pm$ 0.035	0.427 $\pm$ 0.149	9.700 $\pm$ 0.038	0.575 $\pm$ 0.034	0.417 $\pm$ 0.131	9.983 $\pm$ 0.228	0.522 $\pm$ 0.022	0.430 $\pm$ 0.086	6.156 $\pm$ 0.115	0.589 $\pm$ 0.025	0.456 $\pm$ 0.033	28.113 $\pm$ 0.206	0.642 $\pm$ 0.014	0.433 $\pm$ 0.034	13.488 $\pm$ 0.039
cWGAN-GP $\rightarrow$ TVGL	0.678 $\pm$ 0.042	0.570 $\pm$ 0.030	2.602 $\pm$ 0.099	0.520 $\pm$ 0.037	0.513 $\pm$ 0.026	3.498 $\pm$ 0.213	0.507 $\pm$ 0.035	0.500 $\pm$ 0.018	2.018 $\pm$ 0.234	0.510 $\pm$ 0.018	0.477 $\pm$ 0.050	8.523 $\pm$ 0.177	0.554 $\pm$ 0.008	0.515 $\pm$ 0.027	4.160 $\pm$ 0.075
EEG-GAN $\rightarrow$ TVGL	0.375 $\pm$ 0.088	0.335 $\pm$ 0.003	8.288 $\pm$ 0.458	0.537 $\pm$ 0.052	0.373 $\pm$ 0.068	12.802 $\pm$ 0.561	0.547 $\pm$ 0.051	0.391 $\pm$ 0.056	4.474 $\pm$ 0.255	0.566 $\pm$ 0.053	0.417 $\pm$ 0.066	5.792 $\pm$ 0.127	0.507 $\pm$ 0.044	0.379 $\pm$ 0.015	7.839 $\pm$ 0.167

Table 9: Table 2 with standard deviations. Three-seed forecasting performance on EEG datasets, mean $\pm$ standard deviation across three random seeds. Lower is better for AIRM and log-Euclidean RMSE (logE). Both TVGL–CFM couplings are reported: the mean-centred residual coupling and the random-walk coupling. The remaining learned forecasters predict the absolute future directly. Best in **bold**, second best *underlined and italicized*.

Method	Zhou2016		BNCI2014_001		BNCI2014_002		BNCI2015_001		Mean	
	AIRM	logE	AIRM	logE	AIRM	logE	AIRM	logE	AIRM	logE
TVGL–CFM (mean-residual)	2.4154$\pm$0.0211	2.3006$\pm$0.0231	2.7476 $\pm$ 0.0206	2.5449 $\pm$ 0.0176	1.5527$\pm$0.0115	1.4171$\pm$0.0049	<u>1.9529$\pm$0.0085</u>	<u>1.7937$\pm$0.0140</u>	2.1671$\pm$0.0050	2.0141$\pm$0.0059
TVGL–CFM (random-walk)	2.9374 $\pm$ 0.0341	2.7732 $\pm$ 0.0340	3.4454 $\pm$ 0.0205	3.1494 $\pm$ 0.0228	1.8172 $\pm$ 0.0094	1.6367 $\pm$ 0.0061	2.1673 $\pm$ 0.0297	1.9797 $\pm$ 0.0186	2.5918 $\pm$ 0.0083	2.3847 $\pm$ 0.0063
TVGL–cVAE-Forecast	<u>2.4523$\pm$0.0028</u>	<u>2.3379$\pm$0.0017</u>	<u>2.7067$\pm$0.0185</u>	2.5031$\pm$0.0202	1.6017 $\pm$ 0.0039	1.4564 $\pm$ 0.0044	1.9776 $\pm$ 0.0084	1.8096 $\pm$ 0.0081	2.1846 $\pm$ 0.0049	2.0268 $\pm$ 0.0051
TVGL–DDPM-Forecast	2.7683 $\pm$ 0.0080	2.6872 $\pm$ 0.0093	3.3616 $\pm$ 0.0015	3.2063 $\pm$ 0.0009	1.7975 $\pm$ 0.0021	1.6600 $\pm$ 0.0019	2.1407 $\pm$ 0.0027	2.0007 $\pm$ 0.0032	2.5170 $\pm$ 0.0025	2.3885 $\pm$ 0.0028
DyGraphformer-T	2.4606 $\pm$ 0.0077	2.3487 $\pm$ 0.0071	2.6960$\pm$0.0252	<u>2.5074$\pm$0.0273</u>	<u>1.5780$\pm$0.0112</u>	<u>1.4418$\pm$0.0110</u>	1.9404$\pm$0.0041	1.7803$\pm$0.0044	<u>2.1687$\pm$0.0076</u>	<u>2.0196$\pm$0.0067</u>
Training mean (no flow)	2.8473 $\pm$ 0.0000	2.7690 $\pm$ 0.0000	3.2869 $\pm$ 0.0000	3.1430 $\pm$ 0.0000	1.8397 $\pm$ 0.0000	1.7075 $\pm$ 0.0000	2.2037 $\pm$ 0.0000	2.0674 $\pm$ 0.0000	2.5444 $\pm$ 0.0000	2.4217 $\pm$ 0.0000
Persistence	3.0118 $\pm$ 0.0000	2.8219 $\pm$ 0.0000	3.3624 $\pm$ 0.0000	3.0462 $\pm$ 0.0000	1.9314 $\pm$ 0.0000	1.7206 $\pm$ 0.0000	2.5205 $\pm$ 0.0000	2.2729 $\pm$ 0.0000	2.7065 $\pm$ 0.0000	2.4654 $\pm$ 0.0000
Basic-CNN $\rightarrow$ TVGL	9.8436 $\pm$ 0.0057	9.8849 $\pm$ 0.0058	10.8978 $\pm$ 0.0078	10.9431 $\pm$ 0.0078	5.9209 $\pm$ 0.0004	5.9468 $\pm$ 0.0004	8.7390 $\pm$ 0.0002	8.7649 $\pm$ 0.0002	8.8503 $\pm$ 0.0026	8.8849 $\pm$ 0.0026
cVAE $\rightarrow$ TVGL	8.3927 $\pm$ 0.0492	8.4227 $\pm$ 0.0506	9.0986 $\pm$ 0.0752	9.1200 $\pm$ 0.0768	3.1431 $\pm$ 0.0753	3.0970 $\pm$ 0.0780	6.0469 $\pm$ 0.0179	6.0299 $\pm$ 0.0187	6.6703 $\pm$ 0.0160	6.6674 $\pm$ 0.0156
cWGAN-GP $\rightarrow$ TVGL	9.8098 $\pm$ 0.0313	9.8512 $\pm$ 0.0313	10.7351 $\pm$ 0.0226	10.7808 $\pm$ 0.0226	4.9029 $\pm$ 0.0487	4.9258 $\pm$ 0.0475	5.3428 $\pm$ 0.1482	5.2790 $\pm$ 0.1606	7.6977 $\pm$ 0.0590	7.7092 $\pm$ 0.0614
EEG-GAN $\rightarrow$ TVGL	4.3273 $\pm$ 0.0068	4.1014 $\pm$ 0.0255	5.1175 $\pm$ 0.1299	4.7425 $\pm$ 0.1271	2.9781 $\pm$ 0.1515	2.7737 $\pm$ 0.1410	3.1935 $\pm$ 0.0302	2.9720 $\pm$ 0.0315	3.9041 $\pm$ 0.0232	3.6474 $\pm$ 0.0224
JET $\rightarrow$ TVGL	3.8611 $\pm$ 0.0887	3.4902 $\pm$ 0.0778	4.1264 $\pm$ 0.2180	3.8915 $\pm$ 0.2347	3.9463 $\pm$ 0.0314	3.0594 $\pm$ 0.0519	4.4094 $\pm$ 0.3417	3.8906 $\pm$ 0.4006	4.0858 $\pm$ 0.0554	3.5829 $\pm$ 0.0687

N Full chaotic-systems forecasting results

Tables 10–12 give the complete forecasting results for the three chaotic systems of Section 4.3, for a fixed history of $K = 15$ windows and forecast horizons $h \in \{3, 5, 7\}$. Table 3 in the main paper averages over all nine system $\times$ horizon settings.

O Compute

All experiments were run on a single NVIDIA L40 GPU. TVGL target estimation was batched across sequences and windows per dataset. Each TVGL–CFM generator or forecaster trained in under one hour per dataset, while sampling a full forecast ensemble required only a few seconds per batch.

P Use of Large Language Models

Large language models were used to assist with notation refinement, formatting, proofreading, clarity of presentation, figure design, and preliminary proof checking. The authors reviewed, verified, and, where necessary, revised all generated suggestions before inclusion in the manuscript. All scientific claims, mathematical arguments, experimental choices, implementations, and reported results remain the responsibility of the authors.

Table 10: Lorenz forecasting results for $K = 15$ and $h \in \{3, 5, 7\}$. Lower is better. Results are mean $\pm$ std across three training seeds.

Method	Metric	$h = 3$	$h = 5$	$h = 7$
TVGL-CFM (random-walk)	AIRM	1.418$\pm$0.036	<u>1.465$\pm$0.036</u>	1.512 $\pm$ 0.037
	RMSE	1.548$\pm$0.036	<u>1.646$\pm$0.033</u>	1.714 $\pm$ 0.044
TVGL-CFM (train-mean residual)	AIRM	1.646 $\pm$ 0.044	1.706 $\pm$ 0.040	1.757 $\pm$ 0.093
	RMSE	1.816 $\pm$ 0.040	1.929 $\pm$ 0.051	2.007 $\pm$ 0.101
TVGL-cVAE-Forecast	AIRM	1.558 $\pm$ 0.021	1.568 $\pm$ 0.015	1.593 $\pm$ 0.008
	RMSE	1.624 $\pm$ 0.019	1.679 $\pm$ 0.012	1.746 $\pm$ 0.009
TVGL-DDPM-Forecast	AIRM	1.472 $\pm$ 0.021	1.453$\pm$0.002	1.449$\pm$0.007
	RMSE	1.608 $\pm$ 0.019	1.658 $\pm$ 0.007	<u>1.681$\pm$0.010</u>
DyGraphformer-T	AIRM	<u>1.461$\pm$0.021</u>	1.648 $\pm$ 0.020	1.747 $\pm$ 0.013
	RMSE	<u>1.578$\pm$0.023</u>	1.804 $\pm$ 0.022	1.929 $\pm$ 0.015
Global train-mean future	AIRM	1.520 $\pm$ 0.000	1.510 $\pm$ 0.000	<u>1.500$\pm$0.000</u>
	RMSE	1.594 $\pm$ 0.000	1.626$\pm$0.000	1.643$\pm$0.000
Mean train displacement	AIRM	1.991 $\pm$ 0.000	2.005 $\pm$ 0.000	2.003 $\pm$ 0.000
	RMSE	2.105 $\pm$ 0.000	2.167 $\pm$ 0.000	2.197 $\pm$ 0.000
Persistence	AIRM	1.991 $\pm$ 0.000	2.005 $\pm$ 0.000	2.003 $\pm$ 0.000
	RMSE	2.105 $\pm$ 0.000	2.166 $\pm$ 0.000	2.196 $\pm$ 0.000
Linear history-drift	AIRM	3.013 $\pm$ 0.000	3.595 $\pm$ 0.000	4.213 $\pm$ 0.000
	RMSE	3.152 $\pm$ 0.000	3.810 $\pm$ 0.000	4.501 $\pm$ 0.000
Basic-CNN $\rightarrow$ TVGL	AIRM	2.175 $\pm$ 0.035	2.202 $\pm$ 0.008	2.243 $\pm$ 0.024
	RMSE	2.282 $\pm$ 0.044	2.337 $\pm$ 0.008	2.392 $\pm$ 0.029
LSTM $\rightarrow$ TVGL	AIRM	1.555 $\pm$ 0.131	3.727 $\pm$ 0.510	5.050 $\pm$ 0.876
	RMSE	2.059 $\pm$ 0.184	4.483 $\pm$ 0.495	5.536 $\pm$ 0.689
cVAE $\rightarrow$ TVGL	AIRM	3.640 $\pm$ 0.095	3.658 $\pm$ 0.082	3.629 $\pm$ 0.085
	RMSE	3.761 $\pm$ 0.096	3.804 $\pm$ 0.084	3.787 $\pm$ 0.088
cWGAN-GP $\rightarrow$ TVGL	AIRM	2.231 $\pm$ 0.190	2.256 $\pm$ 0.235	2.287 $\pm$ 0.260
	RMSE	2.326 $\pm$ 0.214	2.415 $\pm$ 0.250	2.465 $\pm$ 0.334

Table 11: MacArthur forecasting results for $K = 15$ and $h \in \{3, 5, 7\}$. Lower is better.

Method	Metric	$h = 3$	$h = 5$	$h = 7$
TVGL-CFM (random-walk)	AIRM	0.880 ± 0.007	1.047 ± 0.004	1.077 ± 0.018
	RMSE	0.908 ± 0.009	1.075 ± 0.004	1.101 ± 0.017
TVGL-CFM (train-mean residual)	AIRM	0.894 ± 0.021	1.087 ± 0.025	1.122 ± 0.003
	RMSE	0.930 ± 0.021	1.112 ± 0.019	1.144 ± 0.001
TVGL-cVAE-Forecast	AIRM	0.982 ± 0.008	<u>1.028 ± 0.004</u>	<u>1.058 ± 0.002</u>
	RMSE	0.994 ± 0.007	1.045 ± 0.004	<u>1.078 ± 0.003</u>
TVGL-DDPM-Forecast	AIRM	1.075 ± 0.002	1.071 ± 0.001	1.075 ± 0.002
	RMSE	1.085 ± 0.002	1.087 ± 0.001	1.093 ± 0.001
DyGraphformer-T	AIRM	<u>0.879 ± 0.009</u>	1.019 ± 0.001	1.117 ± 0.010
	RMSE	<u>0.904 ± 0.011</u>	<u>1.051 ± 0.002</u>	1.150 ± 0.010
Global train-mean future	AIRM	1.058 ± 0.000	1.055 ± 0.000	1.053 ± 0.000
	RMSE	1.066 ± 0.000	1.067 ± 0.000	1.067 ± 0.000
Mean train displacement	AIRM	1.568 ± 0.000	1.539 ± 0.000	1.513 ± 0.000
	RMSE	1.572 ± 0.000	1.558 ± 0.000	1.539 ± 0.000
Persistence	AIRM	1.568 ± 0.000	1.539 ± 0.000	1.513 ± 0.000
	RMSE	1.572 ± 0.000	1.558 ± 0.000	1.539 ± 0.000
Linear history-drift	AIRM	2.244 ± 0.000	2.644 ± 0.000	3.051 ± 0.000
	RMSE	2.242 ± 0.000	2.680 ± 0.000	3.126 ± 0.000
Basic-CNN $\rightarrow$ TVGL	AIRM	2.310 ± 0.029	2.406 ± 0.054	2.465 ± 0.023
	RMSE	2.386 ± 0.039	2.506 ± 0.055	2.565 ± 0.027
LSTM $\rightarrow$ TVGL	AIRM	0.706 ± 0.051	1.456 ± 0.093	2.214 ± 0.140
	RMSE	0.791 ± 0.059	1.695 ± 0.110	2.622 ± 0.179
cVAE $\rightarrow$ TVGL	AIRM	4.777 ± 0.029	5.259 ± 0.016	5.434 ± 0.014
	RMSE	4.851 ± 0.024	5.344 ± 0.014	5.522 ± 0.014
cWGAN-GP $\rightarrow$ TVGL	AIRM	2.501 ± 0.273	2.264 ± 0.193	2.381 ± 0.103
	RMSE	2.558 ± 0.300	2.344 ± 0.201	2.505 ± 0.160

Table 12: Hopfield forecasting results for $K = 15$ and $h \in \{3, 5, 7\}$. Lower is better.

Method	Metric	$h = 3$	$h = 5$	$h = 7$
TVGL-CFM (random-walk)	AIRM	0.626±0.002	0.759±0.015	<u>0.832±0.005</u>
	RMSE	0.706±0.006	0.848±0.016	<u>0.911±0.006</u>
TVGL-CFM (train-mean residual)	AIRM	<u>0.636±0.026</u>	0.790±0.034	0.858±0.015
	RMSE	<u>0.724±0.026</u>	0.907±0.039	0.983±0.018
TVGL-cVAE-Forecast	AIRM	0.747±0.004	0.801±0.003	0.830±0.002
	RMSE	0.794±0.005	<u>0.851±0.001</u>	0.883±0.002
TVGL-DDPM-Forecast	AIRM	0.845±0.021	0.890±0.015	0.908±0.014
	RMSE	0.919±0.017	0.961±0.015	0.982±0.012
DyGraphformer-T	AIRM	0.667±0.006	<u>0.784±0.008</u>	0.850±0.015
	RMSE	0.727±0.006	0.857±0.009	0.931±0.012
Global train-mean future	AIRM	0.905±0.000	0.900±0.000	0.899±0.000
	RMSE	0.935±0.000	0.937±0.000	0.939±0.000
Mean train displacement	AIRM	1.289±0.000	1.289±0.000	1.270±0.000
	RMSE	1.342±0.000	1.351±0.000	1.339±0.000
Persistence	AIRM	1.289±0.000	1.288±0.000	1.270±0.000
	RMSE	1.342±0.000	1.350±0.000	1.339±0.000
Linear history-drift	AIRM	1.796±0.000	2.110±0.000	2.409±0.000
	RMSE	1.827±0.000	2.170±0.000	2.495±0.000
Basic-CNN → TVGL	AIRM	1.840±0.029	2.012±0.021	2.082±0.043
	RMSE	1.977±0.032	2.186±0.026	2.269±0.064
LSTM → TVGL	AIRM	0.790±0.036	1.041±0.131	1.626±0.052
	RMSE	0.903±0.039	1.204±0.168	1.866±0.065
cVAE → TVGL	AIRM	2.912±0.122	3.356±0.084	3.649±0.060
	RMSE	3.052±0.120	3.529±0.076	3.839±0.058
cWGAN-GP → TVGL	AIRM	1.690±0.409	1.851±0.633	1.786±0.365
	RMSE	1.769±0.364	2.064±0.925	1.978±0.538